

# iCoder-27B

## Recursive AI-Led Development of Frontier Industrial Coding Model

Core Contributors: Cheng Yang\*, Jiayang Lyu\*, Shangyuan Liu\*; 贡献者: Jiong Lin\*, Xinlei Yu\*

Project Lead: Guibin Zhang\*; Senior Advisors: Junchi Yan\*, Shuicheng Yan\*, Weinan E\*,

Corresponding Authors: Linfeng Zhang\*, Linfeng Zhang\*, Qibing Ren\*

\*School of Artificial Intelligence, Shanghai Jiao Tong University

\*DP Technology \*National University of Singapore †Endless Frontier

### Abstract

递归式人工智能（Recursive AI），即人工智能在构建和改进人工智能方面发挥日益完整作用的前景，是人工智能服务于人工智能的皇冠明珠。尽管递归式自我发展对于小模型、有界任务和固定时间预算已变得实用，但这一雄心壮志的更重大实现，即开发一个可发布、具备前沿竞争力的模型，仍然更具挑战性。在这项工作中，我们探讨了智能体开发前沿模型所需的最少人类参与程度。我们将人类输入集中到一个高密度、低频率的接口中：专家将目标、阶段脚手架、权限边界和操作流程编码为可复用的研究技能，而智能体则实例化这些先验知识、选择实验、诊断结果并修订训练策略。在具有挑战性的工业编码领域，智能体演化数据并协调监督微调（SFT）、在策略自蒸馏以及具有可验证奖励的强化学习，最终产生了 iCoder，一个用于寄存器传输级（RTL）设计和图形处理器（GPU）核函数优化的 27B 模型。在七个基准测试中，iCoder 在 RTLLM 上领先，超越了 GPT-5.5 和 Claude-Opus-4.8；在 CVDP 和 KernelBench L2 上排名第二，超过 GPT-5.5 达 16 分；并在 TritonBench 上并列 Claude-Opus-4.8 的最佳结果。探索性案例研究进一步表明，iCoder 在迭代式 RTL 和 GPU 核函数优化方面具有竞争力，且使用的令牌数量大幅减少。这些结果为通向递归式自我改进的工程路径指明了方向：人类提炼模型构建的原则，智能体通过证据驱动的实验将其操作化，每一代人工智能都成为下一代更具能力的架构师。

GitHub: <https://github.com/bingreeky/iCoder>

Hugging Face: <https://huggingface.co/i-Coder>

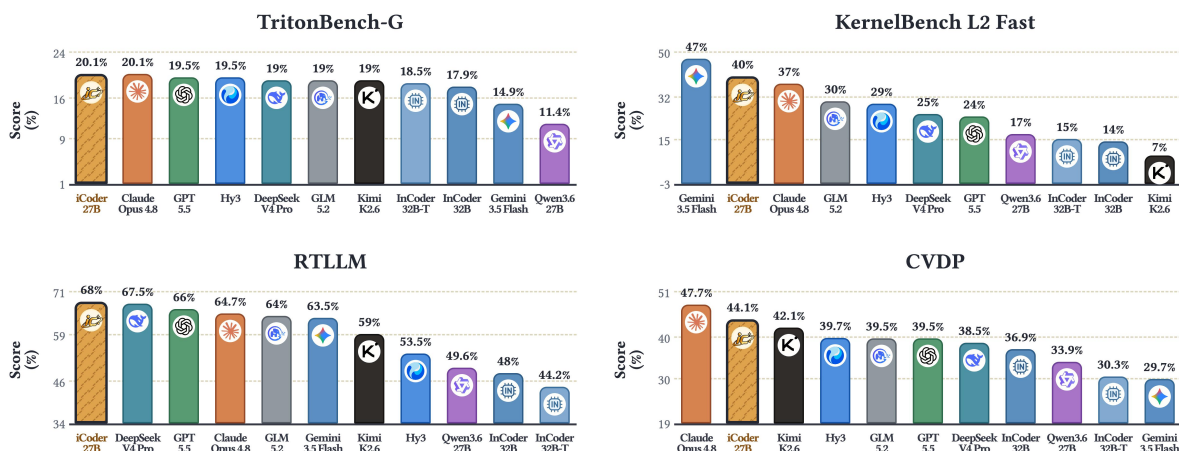


图 1：跨核函数和 RTL 基准的工业编码性能。我们将 iCoder 与领先的开源和专有模型在 TritonBench-G、KernelBench L2 Fast、RTLLM 和 CVDP 上进行比较。每个基准内的条形图独立排序，分数越高越好。尽管 iCoder 规模紧凑，仅为 27B，但它在 KernelBench L2 Fast 和 CVDP 上排名第二，在 RTLLM 上领先，并在 TritonBench-G 上并列最佳结果。

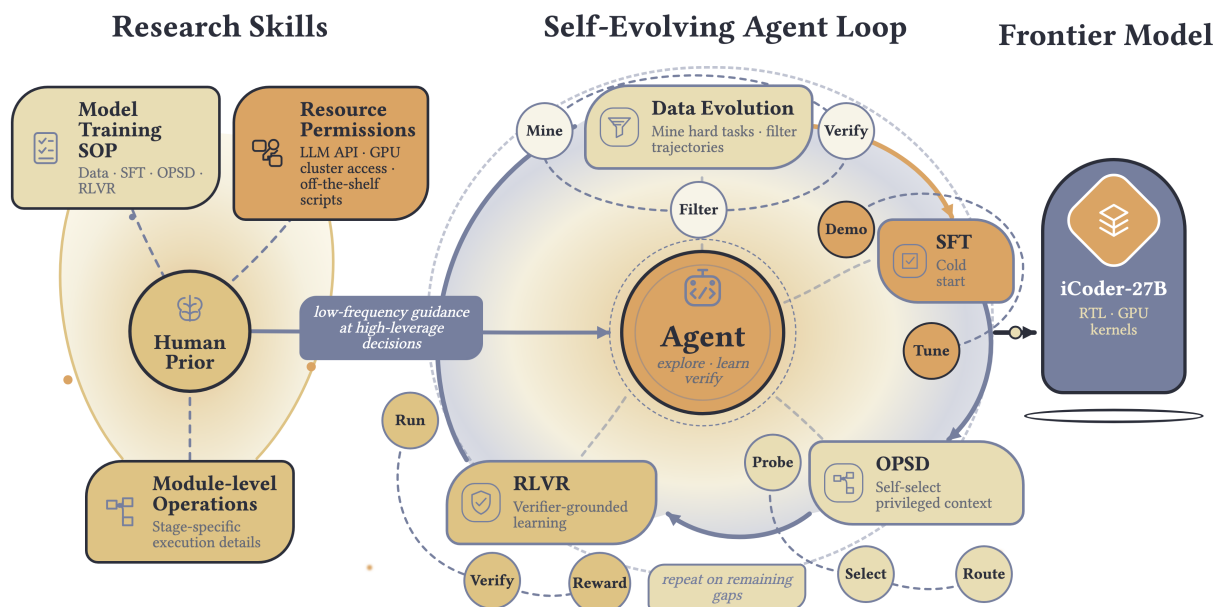


图 2：我们的人机协同自进化训练框架概览。研究技能被提炼为模型训练的标准作业程序、资源权限和模块级操作，在杠杆决策点提供低频人类指导。随后，智能体自主迭代数据进化、监督微调、离策略采样蒸馏和基于人类反馈的强化学习，以弥合剩余能力差距，并产出前沿模型 iCoder-27B。

## 1 引言

人工智能智能体正从辅助孤立的科研任务，迈向执行日益完整的科学 workflows（Yamada 等，2026；Zhao 等，2026a）。近期系统能够将一个人工智能项目从假设形成推进到实验和论文撰写（Yamada 等，2026），在量化反馈下搜索可执行的解空间（Lin 等，2026），并通过持久状态和自适应算力分配维持长周期研究（Zhao 等，2026a）。这些进展重新点燃了对递归自我改进的兴趣，但最强有力的证据仍来自外部提供的评估器所约束的有界循环（Chen 等，2026；Ding 等，2026）。以下问题依然具有挑战性：

### Research Question

*Can AI coordinate the development of a genuinely competitive frontier model?*

前沿模型的开发比改进单个程序、测试框架或基准分数提出了更高的要求。近期的递归自我改进系统在封闭沙箱内迭代，其数据、评估器和动作空间是固定的（Guo 等，2026），或是在合成研究任务上优化提示级测试框架（Lee 等，2026）。即使一项跨越约一百次实验的长期研究，也仍局限于单一架构问题，并依赖人工声明的阶段转换（Safdar 和 Saadeldin，2026）。相比之下，开发一个可发布的基础模型需要一个紧密耦合的技术栈：构建数据时的选择会重塑分布式训练、后训练、评估，并最终影响下一轮决策（Qwen Team，2026；DeepSeek-AI，2026；Li 等，2026）。实验成本高昂，失败会在各阶段间传播，且其原因很少能被局部定位。因此，核心问题不仅仅是智能体能否优化，而是它必须接收哪些专家知识、以何种粒度接收，以及人类需要以多高的频率重新介入循环。

本文并不主张直接跃升至完全封闭的递归自我改进（Recursive Self-Improvement）。相反，本文研究一个实用的中间目标：在开发一个可发布、具备前沿竞争力的模型过程中，人类参与程度最低可以降到多少？本文的答案是高密度先验、低频干预。本文编码

专家模型开发经验曾作为人类撰写的研究技能：简洁的操作流程，辅以可复用的代码，使既定实践可在图形处理器集群上执行。这些技能将数据准备、分布式监督微调、离策略采样蒸馏和基于可验证奖励的强化学习中的经验转化为可用的起始条件，而不预设后续实验。自此之后，智能体自行选择如何实例化这些技能、分配资源、运行并诊断实验，并决定下一步尝试。因此，最终模型由人类与智能体共同贡献，但存在明确边界：人类提供初始可执行的专业知识，而具体的模型开发决策与执行则属于智能体。我们研究这种分工在工业编码中的应用，涵盖寄存器传输级设计与图形处理器核函数优化，其中成功既需要精确的功能推理，也需要在可执行验证下进行性能感知的系统优化。在研究技能就绪后，我们将开发循环交由以极高推理努力运行的 Codex GPT-5.6-Sol。从基础检查点与可用算力出发，该智能体①构建了可执行任务池，②在验证的能力差距轨迹上冷启动 Qwen3.6-27B，随后以离策略采样蒸馏作为通往基于可验证奖励的强化学习的桥梁。在③离策略采样蒸馏期间，评估改变了进入训练的数据，受控消融改变了自教师可访问的证据，而坍塌的目标迫使智能体围绕验证器锚定的信用重新设计学习规则。在④基于可验证奖励的强化学习期间，奖励利用、错误判定和长度依赖的退化相继改变了奖励资格、优化和轨迹预算。智能体并非执行固定的人类撰写配方，而是通过这些失败与修订最终得出最终配方。该过程最终产出了一个 27B 工业编码模型，我们称之为 iCoder-27B (iCoder)。如图 1 所总结，尽管其尺寸紧凑，iCoder 与更大的开放模型及前沿专有系统相比极具竞争力。与同尺寸的 Qwen3.6-27B (Qwen 团队, 2026) 相比，它在 RTLLM 上提升 18.4 分 (68.0 对 49.6)，在 VerilogEval 规格到寄存器传输级上提升 16.2 分 (86.3 对 70.1)，并达到  $2.64\times$  的 KernelBench L2 正确率 (74 对 28)。它还超越了总参数 1.6T/活跃参数 49B 的 DeepSeek-V4-Pro (DeepSeek-AI, 2026) 在 RTLLM 上的表现 (68.0 对 67.5)，并几乎将其 KernelBench L1 正确率翻倍 (61 对 32)。更引人注目的是，iCoder 在 RTLLM (68.0 对 66.0/64.7/63.5) 和 KernelBench L1 正确率 (61 对 43/55/45) 上均优于 GPT-5.5 (OpenAI, 2026)、Claude Opus 4.8 (Anthropic, 2026) 和 Gemini 3.5 Flash (Google DeepMind, 2026)。在 TritonBench-G 上，它与 Claude Opus 4.8 持平于 20.1，同时分别超过 GPT-5.5 和 Gemini 3.5 Flash 的 19.5 和 14.9。除单次生成外，评估驱动的案例研究表明，iCoder 在八个寄存器传输级设计上与 HY3 和 DeepSeek-V4-Pro 的差距分别仅为 1.1 和 4.5 分，同时报告的输出令牌数分别仅为后者的 51% 和 33%，并在八个选定的图形处理器算子中的两个上达到了最强的存档终点。我们的贡献有三方面：• 我们将前沿模型开发表述为智能体自我改进的真实世界测试，并提供了初始人类专业知识与智能体主导开发之间的明确归因边界。

- 我们将专家知识操作化为研究技能，并记录一条可审计的实验轨迹，其中数据、目标、奖励语义和资源约束会根据证据进行修订。
- 我们交付了 iCoder-27B，这是一个紧凑的 27B 模型，在工业编码基准上可与或超过规模大得多的开放和专有模型。

## 2 相关工作

**递归与自我改进的人工智能。**近期的自我改进人工智能系统最好通过每次迭代中修订的内容来区分：外部解决方案、围绕模型的智能体机制，或模型本身 (Chen 等, 2026; Ding 等, 2026; Yamada 等, 2026; Zhao 等, 2026a; Luo, 2026)。在第一层，固定模型在评估器反馈下反复提出并测试候选解决方案；

改进累积在生成的工件中，而非产生该工件的模型中（Novikov 等，2025；Assumpção 等，2025；Du 等，2026；Guo 等，2026；Lin 等，2026）。递归也可以进入智能体框架，使执行轨迹改变冻结模型在后续任务上的行为（Lee 等，2026；Wang 等，2026b；Zhou，2026；Alzubi 等，2026）。这提升了系统级性能，但仍受底层模型能力的限制（Lee 等，2026；Wang 等，2026b；Zhou，2026；Alzubi 等，2026）。更强的形式通过在自生成轨迹和可验证结果上训练，将经验写回模型参数（Acikgoz 等，2025；Zhao 等，2026b；Hübötter 等，2026；Yang 等，2026；Rank 等，2026）。只有最后一种形式使获得的能力独立于特定框架而持续存在，因此最直接地接近自主模型开发（Chen 等，2026；Yamada 等，2026；Acikgoz 等，2025；Rank 等，2026）。然而，这一层面的证据仍受到固定任务接口、短预算或有限问题规模的严格限制（Rank 等，2026；Du 等，2026；Safdar 和 Saadeldin，2026；Guo 等，2026）：直接的智能体后训练主要在 4B 级模型上研究，每次运行限于单个 H100 上十小时，并针对一个目标基准进行优化（Rank 等，2026）。我们的工作在不同的操作规模上研究相同的模型级目标，要求智能体协调一个多阶段过程，其最终工件是一个可发布的 27B 模型。

**工业代码智能。**本文使用工业编码来指代那些验收由专用可执行工具链和部署质量决定、而非仅由表面相似性决定的代码生成问题（Pinckney 等，2025；Fu 等，2026；Ouyang 等，2025；Li 等，2025；Zang 等，2026；Huang 等，2026）。在硬件设计中，寄存器传输级生成是更广泛电子设计自动化工作流程中代码密集的阶段：行为规范必须转化为能够展开、综合并满足其预期行为的时钟电路描述（Liu 等，2023；Lu 等，2023；Pinckney 等，2025；Fu 等，2026；Deng 等，2026）。因此，评估已从短小、孤立的模块发展到现实规范、层次化知识产权设计以及基于仿真、形式化和综合的验证（Liu 等，2023；Lu 等，2023；Pinckney 等，2025；Jin 等，2025；Purini 等，2025；Fu 等，2026；Wang 等，2026a）。图形处理器核函数生成构成一个并行的系统编码领域，而非电子设计自动化的另一个阶段（Ouyang 等，2025；Li 等，2025；Jaber 和 Jaber，2026；Brodsky 等，2026）。在此领域中，程序必须能够编译、保持数值语义、高效利用目标机器，并提供实测加速（Ouyang 等，2025；Li 等，2025；Jaber 和 Jaber，2026；Brodsky 等，2026；Zang 等，2026；Huang 等，2026）。因此，近期的评估已从孤立的算子翻译扩展到多样化工作负载、硬件可移植性以及集成到真实软件框架中（Ouyang 等，2025；Li 等，2025；Jaber 和 Jaber，2026；Brodsky 等，2026；Zang 等，2026；Huang 等，2026）。这两个领域测试互补的能力：寄存器传输级在精确行为约束下强调时序和层次推理，而核函数在数值约束下强调并行化和性能推理（Jin 等，2025；Purini 等，2025；Fu 等，2026；Ouyang 等，2025；Zang 等，2026；Huang 等，2026）。大多数先前系统孤立地研究其中一个领域（Deng 等，2026；Jaber 和 Jaber，2026；Brodsky 等，2026）；本文同时使用两者来评估单一模型能否支持更广泛的工业代码智能概念。

### 3 人类先验：可执行的研究技能

#### Design Principle: The Bitter Lesson for Model Development

萨顿的“苦涩教训”指出，当计算支持搜索与学习而非人工设计的决策时，通用方法能够扩展（萨顿，2019）。我们将这一原则应用于模型开发：人类将目标、可信验证器、权限和可复现性编码为可执行的过程先验。在此边界内，智能体自身利用证据来选择数据、目标、奖励和阶段转换。



### 3.1 从专家经验到可执行先验

我们将人类先验表示为一组版本化的研究技能集合：将宽泛的模型开发目标转化为可执行但有意欠规范的研究程序的操作性知识。这些技能编码了经验丰富的团队通常通过反复干预来传达的程序，包括如何访问已批准的资源、构建可审计的训练工件、启动分布式作业、诊断故障以及使用可执行验证。它们还定义了不得通过试错学习的不变量，例如留出数据分离、验证器完整性、权限边界，以及每个报告的检查点都必须可追溯到其数据、代码、配置和父模型的要求。表 1 总结了这一先验。该先验固定了目标、阶段脚手架、可接受的动作空间，以及模型、仓库、数据源、启动器、验证器配置文件和资源配额的运行时绑定。智能体选择数据变换、特权上下文、优化目标、奖励适配器和检查点。实验发现进入实验记录并为后续决策提供信息。

**表 1：人类先验的范围。**研究技能指定可复用的程序和不可协商的边界，同时将经验选择留给智能体。运行特定的身份和测量发现分别记录在项目状态中。

| 先验层 人类指定的内容 智能体动作的边界                                                                                                              |                                                                                                                |                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| 🎯 Objective and scaffold                                                                                                          | Target capability, completion criteria, and the Data-SFT-OPSD-RLVR stage roles                                 | May select experiments and transitions, but may not silently redefine the deliverable.                         |
| <b>Skill excerpt — stage controller</b><br>data → SFT → OPSD → RLVR<br>↔ returning to data is normal                              |                                                                                                                |                                                                                                                |
| 📦 Resource access                                                                                                                 | Approved repositories, models, data, launchers, compute, storage, and external-side-effect permissions         | May use registered capabilities within budget; new identities or permissions require a human gate.             |
| ✅ Verification                                                                                                                    | Task-contract schema, official-harness requirement, integrity checks, and pass/fail/unjudged semantics         | May integrate and audit a verifier profile; may not weaken or replace the approved correctness criterion.      |
| 🔧 Research method                                                                                                                 | Controlled pilots, explicit baselines, artifact lineage, stop rules, and regression-aware checkpoint selection | May formulate hypotheses and choose the next discriminating test; conclusions must cite registered evidence.   |
| 📅 Governance and memory                                                                                                           | Task Queue, experiment log, decision records, and approval boundaries                                          | May write run-scoped findings and negative results; the released Human Prior remains immutable during the run. |
| <b>Condensed executable rule — decision memory</b><br>observe → conclude → act → expect<br>return → record what actually happened |                                                                                                                |                                                                                                                |

### 3.2 面向后训练的分阶段先验

该框架将可执行数据构建与三个参数更新阶段分离。数据构建首先产生任务-预言机对，其正确性可被测试，因为后续每个阶段都需要关于生成代码的可靠信号。监督微调随后从经过验证的教师轨迹中提供推理冷启动，这些轨迹针对暴露模型相对能力差距的任务。在线策略自蒸馏占据中间阶段，在该阶段当前模型只有在获得任务特定指导或反馈后才能解决任务。其预期作用是将观察到的自我纠正转化为持久能力，同时保持部署策略以原始任务为条件。强化学习验证最终将监督移至在线：策略探索自身响应，并根据可执行结果获得任务条件奖励。这些角色决定了顺序 数据 → SFT → OPSD → RLVR。智能体在每个阶段内确定具体干预措施。阶段顺序可修订。监督微调可揭示缺失的任务族，并将循环返回至 5

数据构建。在线策略自蒸馏仅在评估识别出已命名且可恢复的弱点后才进入，并且当无法构建合适的失败经验时可能返回上游。强化学习验证需要可靠的验证器以及具有信息性奖励变化的展开组；否则智能体将修订验证、奖励或数据设计。阶段转换仅在原始检查点及其回归在固定协议下评估后才被接受。

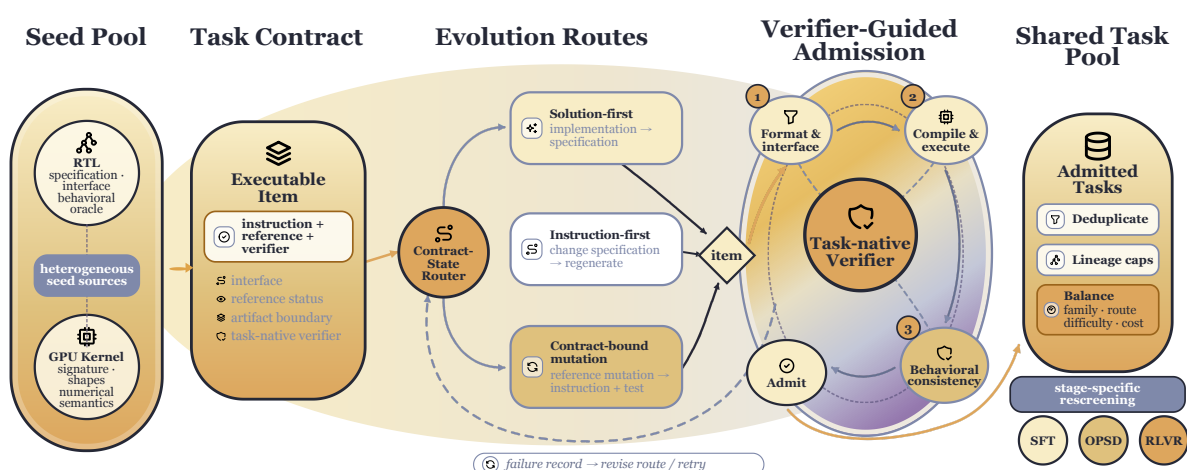
### 3.3 共享资源、治理与记忆

技能通过经批准的项目配置文件公开仓库、模型与数据接口、任务原生验证器、集群启动器、存储、监控及文档流程。资源访问基于能力：智能体可在声明的计算与权限范围内组合并执行已注册组件。新数据集、教师模型、正确性规则、外部副作用及边界变更需经人类授权。数据策略、评估语义与发布权限仍由人类掌控。在此范围内，智能体任务队列使前述内容可执行。每个条目记录研究问题或构建目标、其依赖项、可执行动作、预算、预期工件、验收规则及结果所支持的决策。智能体反复观察当前证据，诊断局限性，提出最小判别实验，执行该实验，验证其工件，并保留、修正或拒绝假设。由此产生的决策与负面结果构成运行范围的研究记忆。该记忆改变下一步尝试的任务，并保留原始人类先验与循环所获知识之间可审计的边界。

## 4 智能体实验轨迹

### 4.1 数据：验证器引导的共享可执行任务池演化

**研究先验与决策空间。**数据研究技能提供了通用可执行条目契约、演化算子库及表 2 总结的准入不变量。人类专家注册种子源与任务原生验证器。试点证据指导智能体的路径选择、验证门顺序、重试预算及最终任务池组成。



**图 3：验证器引导的共享可执行任务池构建。**智能体将 RTL 与 GPU 核函数种子规范化为通用条目契约，在三条演化路径中选择以形成候选任务-预言机对，并应用有序的任务原生验证门。失败更新路径与重试决策；准入条目经去重与平衡后，针对 SFT、OPSD 和 RLVR 进行阶段特定复筛。

**表 2：向智能体公开的数据研究技能。**该技能提供可执行原语与准入不变量，而转换与语料库组成选择仍由证据驱动。

| 技能组件 公开的原语与不变量 智能体决策空间                                                                                                                          |                                                                                                        |                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
|  Task contract                                                 | Specification, generated-artifact boundary, interface, reference status, and task-native harness       | Normalize each seed and determine whether its current oracle is usable.          |
| <b>Condensed executable rule — item contract</b><br>item := instruction + reference solution + verifier<br>require all three parts to agree     |                                                                                                        |                                                                                  |
|  Evolution operators                                           | Behavior-preserving rewriting and reconstruction; contract-bound behavioral mutation                   | Select an operator per seed and revise the operator library from pilot failures. |
| <b>Skill excerpt — solution-first evolution</b><br>mutate reference solution → derive instruction<br>→ generate verifier from the same mutation |                                                                                                        |                                                                                  |
|  Generation roles                                              | Separate proposal, implementation, specification, and audit roles                                      | Decide when a teacher-generated candidate can become a verified reference.       |
|  Admission gates                                               | Interface preservation, leakage checks, semantic-change tests, solvability, compilation, and execution | Order cheap and expensive gates, classify failures, and allocate retries.        |
| <b>Condensed executable rule — rejection pipeline</b><br>static gates → verifier strength → contract audit<br>→ dedup → difficulty → compose    |                                                                                                        |                                                                                  |
|  Pool composition                                              | Ancestor-level split, lineage, deduplication, difficulty measurement, and coverage accounting          | Set per-lineage caps and balance coverage, difficulty, and cost.                 |

**规范化异构种子。**智能体通过自然语言规范、生成工件边界、接口、参考状态和可信验证器来表示每个种子。对于寄存器传输级任务，该契约记录了模块端口、时钟与复位行为、时序要求以及行为预言机。对于图形处理器核函数，该契约记录了模型接口、参数形状、数值语义以及允许的实现边界。这种表示方式使两个领域能够共享规划、溯源和组合逻辑，同时保留任务原生的执行方式。

**根据契约状态进行演化路由。**智能体根据参考可用性以及变换所影响的契约表面，在三个演化族之间路由种子。解优先重构从已验证的实现中推导出新的规范，同时保留其行为和验证器。指令优先演化改变规范，并在验证之前重新生成受影响的执行组件。契约绑定变异将声明的行为算子应用于参考，然后从同一变更中推导出匹配的指令和测试。对于第三条路由，演化后的参考必须通过演化后的测试，而原始参考必须未通过该测试。试点结果根据验证产量、与父代的行为距离、拒绝原因以及每个存活项的成本来确定规模分配。

**失败驱动领域自适应。**寄存器传输级试点表明，压缩后的规范经常遗漏复位、时序或状态转换规则。因此，保留的流水线以机械方式提取接口，保留显式的时钟和复位契约，并对有状态设计使用结构感知算子。核函数试点暴露了另一种模式：一些变换使计算保持不变，一些产生普通的 PyTorch 代码而非 Triton 代码，还有一些在签名或张量形状上发生漂移。保留的核函数路径固定了模型接口和输入契约，检查了实现可行性，并针对可信参考执行候选实现。两条路径都在编译和任务原生执行之前安排了低成本的格式和接口检查。拒绝阶段仍保留在记录中，并指导下一个算子、重试策略和采样分配。附录 A.4 描述了相应的验证器契约。

**组合并移交共享任务池。**经过可执行验证后，智能体对候选方案进行了去重，并分两轮构建任务池。它首先为每个种子谱系选择一个有效候选，然后在每个种子上限内填充剩余预算，同时平衡任务族、路径、难度和验证成本。每个阶段都会根据检查点和监督目标对生成的共享任务池进行重新筛选；附录 A.1 报告了其组成。

4.2 监督微调：从已验证教师轨迹冷启动推理

**研究先验与决策空间。**监督微调研究技能定义了一个经过验证的冷启动任务族。它提供了稳定的提示-响应格式、任务原生轨迹验证、损失掩码和长度审计、生产路径冒烟测试以及组合感知的检查点评估（表 3）。在人工定义的能力差距任务族中，智能体选择了采样预算、学生和教师成功谓词、保留的通过轨迹、分布式布局以及饱和点。这些决策根据当前模型、数据池和计算配置进行了记录。预期结果是在基础模型未显示任何验证成功的任务上实现最小推理冷启动。

表 3：暴露给智能体的监督微调研究技能。先前的修复确定了数据和训练完整性要求，但将模型相关的课程、规模和停止决策留给实验。

| Skill component                                                                                                                                                               | Exposed primitives and invariants                                                                   | Agent decision space                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
|  Format contract                                                                             | Prompt template, response boundary, extraction, tokenizer, and loss mask                            | Audit compatibility and reject format changes that invalidate comparison.       |
|  Capability-gap filter                                                                      | Repeated student and teacher sampling with task-native verification                                 | Choose sampling budgets and the failure/success predicate defining the subset.  |
| <b>Condensed executable rule — model-relative filtering</b><br>student trials $\leftarrow$ sample(student, task)<br>subset $\leftarrow$ select(gap-predicate(student trials)) |                                                                                                     |                                                                                 |
|  Verified target                                                                           | Complete teacher trajectory retained only after an executable pass                                  | Select among passing trajectories and record uncovered tasks.                   |
| <b>Condensed executable rule — target admission</b><br>candidates $\leftarrow$ sample(teacher, task)<br>retain y only if verifier(y) = pass; log uncovered tasks              |                                                                                                     |                                                                                 |
|  Training integrity                                                                        | Length audit, no-truncation rule, production-path smoke test, checkpoint and telemetry requirements | Choose sequence budget, parallel layout, batch configuration, and save cadence. |
| <b>Condensed executable rule — length integrity</b><br>$B \leftarrow$ empirical length distribution of training pairs<br>length > B $\Rightarrow$ drop; never truncate        |                                                                                                     |                                                                                 |
|  Evaluation and routing                                                                    | Fixed protocol, composition-aware metrics, and failure taxonomy                                     | Select the checkpoint, diagnose saturation, and return to Data or enter OPSD.   |

**选择已验证的教师轨迹。**智能体选择了 Qwen3.6-27B pass@4 等于零且在四条 DeepSeek-V4-Pro 轨迹中至少有一次验证成功的任务。每个任务保留一条完整的通过轨迹。在该语料库上进行全参数监督微调，得到了冷启动推理检查点。

4.3 在线策略自蒸馏：验证器锚定自蒸馏的递归发现

**研究先验与决策空间。**在线策略蒸馏在学生生成的轨迹上评估教师监督，减少了离线策略蒸馏的训练-推理分布不匹配（Agarwal 等人，2024）。在线策略自蒸馏将这一思想特化为单一模型，其教师和学生视图仅在接收的上下文上有所不同（Zhao 等人，2026b）。人工提供的技能将这种同权重、双上下文任务族作为监督冷启动与在线学习之间的候选桥梁。



强化学习与可验证奖励（RLVR）。运行时错误和评判反馈可以将当前模型调节为一个反馈知情的自我教师（Hübotter 等，2026）；因此，该技能允许将计划、失败尝试和验证器诊断作为候选证据，但须遵守严格的无解边界。

表 4 明确了归因边界。人类先验要求具备一个命名的可测量弱点、同权重的教师和学生视图、可执行的输出证据、泄漏控制，以及能够区分有效更新与单纯损失下降的检测手段。智能体选择了可恢复性标准、特权视图的内容和粒度、上下文偏好对更新方向和令牌贡献的作用，以及要推进的检查点。受控消融和失败驱动的重新设计解决了这些耦合的决策。

**表 4：暴露给智能体的 OPSD 研究技能。** 先验定义了一个以验证器为锚的自蒸馏家族；数据资格、特权证据和学习规则通过受控测试来选择。

| 技能组件 公开的原语与不变量 智能体决策空间                                                                                                                                                                                       |                                                                                                  |                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| 🎯 Target and data                                                                                                                                                                                            | Named measured weakness, repeated sampling, execution-feedback loop, and recoverability evidence | Define the eligibility predicate and rescreen it under the current checkpoint.         |
| <b>Condensed executable rule — entry contract</b><br>require named weakness + baseline measurement<br>require teacher advantage relevant to that weakness                                                    |                                                                                                  |                                                                                        |
| 👁️ Privileged view                                                                                                                                                                                           | Plan, failed responses, verifier diagnostics, and matched control contexts                       | Select components, ordering, granularity, and stopping point for $c^*$ .               |
| 🔒 Information boundary                                                                                                                                                                                       | No reference answer, code-overlap scan, independent audit, and context integrity                 | Reject leaking plans and decide whether a component remains admissible.                |
| 📊 Learning signal                                                                                                                                                                                            | Frozen privileged scoring, executable outcome, group baseline, and clipped update family         | Decide what determines update direction and how teacher evidence assigns token credit. |
| <b>Skill excerpt — offline OPSD phases</b><br>roll out → score → weight → train<br>generation and teacher scoring remain outside the update phase                                                            |                                                                                                  |                                                                                        |
| 🛠️ Instrumentation                                                                                                                                                                                           | Loss, parameter movement, trajectory behavior, regression matrix, and stop conditions            | Diagnose ineffective or collapsing objectives and revise or reject the intervention.   |
| <b>Condensed executable rule — update validity</b><br>track(relative parameter update, mean log-ratio, clipped-token fraction)<br>assert first-step ratio $\approx 1$ ; re-roll when drift exceeds tolerance |                                                                                                  |                                                                                        |

**选择可恢复的能力前沿。** 智能体首先使用重复采样来识别在当前 SFT 检查点下  $\text{pass}@4$  等于零的任务。然后运行执行反馈循环，并保留那些在初始裸提示尝试中失败但在后续轮次中解决的任务。这种第 0 轮失败且循环解决的标准区分了可恢复的失败与已经掌握的示例，以及重复交互无法提供有用纠正的失败。应用该标准产生了一个包含 1,874 个任务的池。这些任务定义了一个能力前沿，在该前沿上，模型自身的不成功尝试和验证器反馈足以达到解决方案。

**通过受控消融构建特权上下文。** 该智能体将上下文构建视为一个设计问题，包含三个独立的问题：指导是否应针对特定任务、应暴露多少验证器细节，以及在移除答案泄露后所得信号是否仍然有用。表 5 的 (a) 部分列举了所有测试过的上下文视图；(b) 部分仅报告匹配的对比。每个对比使用各自的评估集和主要终点；跨行效应比较无效。此处  $\text{avg}@5$  对每个任务取五个二元正确性结果的平均值，与  $\text{pass}@5$  不同；pp 表示百分点。在相同的 194 个任务和采样预算下，针对特定任务的计划使  $\text{avg}@5$  提高了 6.1 个百分点；在首轮失败的任务上增益为 8.5 个百分点，在初始已解决的任务上增益为 4.0 个百分点。另一项

**表 5：特权上下文构建与匹配消融。** (a) 部分总结了智能体考虑的所有上下文变体以及  $c^*$  的最终构成。(b) 部分报告了计划指导、验证器诊断、反馈粒度和任务-上下文匹配的受控比较。每个比较使用各自的匹配评估集，并应在相应行内进行解释。

| (a) 候选上下文与最终构成 上下文视图 任务提示之外的信息 处置     |                                                                                                                          |                 |  |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------|-----------------|--|
| Bare context $c_\emptyset$            | None; the model receives only the task prompt.                                                                           | Reference       |  |
| Direction-only plan $c_{\text{plan}}$ | A task-specific repair direction; code, assignments, constants, and answer-specific mappings are prohibited.             | Include plan    |  |
| Generic domain hint                   | Task-independent domain advice without evidence from the current attempt.                                                | Exclude         |  |
| Error-conditioned experience          | A failed response paired with its diagnostic verifier output.                                                            | Include signal  |  |
| Coarse experience $c_{\text{coarse}}$ | Failed responses plus verifier stage or mismatch rate and a general repair checklist.                                    | Include         |  |
| Fine experience $c_{\text{fine}}$     | $c_{\text{coarse}}$ plus at most three input-level expected-versus-observed counterexamples.                             | Exclude detail  |  |
| Final $c^*$                           | Audited $c_{\text{plan}}$ plus every failed response and its coarse verifier feedback before the first successful round. | <b>Selected</b> |  |

| (b) Matched comparisons |                                                                    |                                                |                                                                      |
|-------------------------|--------------------------------------------------------------------|------------------------------------------------|----------------------------------------------------------------------|
| Question                | Treatment / control                                                | Evaluation protocol                            | Primary result                                                       |
| Plan value              | $c_{\text{plan}}$ VS. $c_\emptyset$                                | Same 194 tasks; five samples per condition     | avg@5: 47.4% vs. 41.3% (+6.1 pp)                                     |
| Diagnostic value        | Failure history with diagnostics vs. the same history without them | 92 round-0-failure tasks                       | Additional solved tasks: 6/92 (+6.5 pp)                              |
| Feedback detail         | $c_{\text{fine}}$ VS. $c_{\text{coarse}}$                          | Reported 58-task paired contingency            | Final coverage: 49/58 in both conditions (exact McNemar $p = 1.00$ ) |
| Task match              | Matched $c^*$ vs. shuffled $c^*$                                   | Within-domain shuffle; same responses rescored | Mean response log-likelihood: approximately +0.18 nats               |

分布内试点发现，与任务无关的领域提示产生了净负面效应，因此这些提示被排除在默认上下文之外。随后，反馈实验将诊断价值与诊断粒度分离开来。在 92 个任务的首轮失败池中，移除验证器诊断使已解决任务的覆盖范围减少了六个任务。在粒度比较中，细粒度和粗粒度反馈各自解决了所报告的配对列表表中 58 个任务中的 49 个；五个仅细粒度成功和五个仅粗粒度成功的结果得出精确的麦克尼马尔  $p$  值为 1.00。细粒度反馈产生了四次更多的中间循环挽救（23 次对 19 次），但并未扩大最终覆盖范围。因此，该智能体选择了由这些干预支持的最小上下文：

$$c_i^* = c_{\text{plan},i} \oplus \{(y_{i,q}, f_{i,q})\}_{q < q_i^*}, \quad (1)$$

此处  $y_{i,q}$  和  $f_{i,q}$  分别是失败轮次  $q$ ,  $q^*$  的响应和粗粒度验证器反馈， $\oplus$  concatenates 是按顺序排列的上下文；成功响应被省略。

计划组件在进入  $c^*$  之前通过了三道泄漏控制：对解决方案内容的指令级禁止、针对代码和答案特定重叠的基于规则的扫描，以及由不同于计划生成器的模型进行的审计。反复未能通过这些检查的计划被移除，仅保留经过审计的失败经验。最终的无训练信号审计比较了同一响应的四种视图：裸提示  $c_\emptyset$ 、其匹配的  $c^*$ 、域内打乱的  $c^*$ ，以及长度匹配的中性上下文。匹配上下文超出打乱对照约 0.18 纳特。

格式仅占最大令牌级差异的 12 – 22%，而其余差异集中在代码令牌上。该对照排除了上下文长度和格式作为所观察到的教师优势的充分解释。

**间隙导向优化的失败。** 首个实现使用特权视图与裸视图之间的令牌级似然差距作为在线策略更新的方向。由于特权视图由当前行动者评分，其分布随学生移动，形成了自我强化的反馈路径。早期检查点在若干较短视界任务上有所改进，但同一干预使较难任务退化。匹配的 ArchXBench 轨迹长度约变为原来的 2.9 倍，且部分生成反复重现中间推理片段而不输出可执行代码。持续训练最终退化为重复生成。该代理将此失败解释为上下文偏好与环境正确性被混淆的证据。似然差异识别了在  $c^*$  下更受偏好的令牌；可执行结果决定了响应是否应被强化。

#### Agent-Discovered Lesson: Privilege Can Allocate Credit, Not Authority

特权视图揭示了哪些令牌在有用经验下变得更可能，但并未揭示所产生行为是否正确。失败的在线目标暴露了一个清晰的分离：可执行结果必须决定学习方向，而特权上下文只应重新分配令牌级信用。在自蒸馏中，信息优势不等于优化权威。

**将更新方向与令牌信用分离。** 智能体自主地将在线循环替换为离线五阶段流水线：裸提示词展开、在  $c^*$  下的教师强制评分、可执行验证、带符号优势构建以及截断重要性加权更新。冻结并预计算教师评分打破了移动教师反馈路径。对于响应  $i$ ，验证器首先生成原始分层优势  $A_{i,raw}^{OPSD} = A_i^{valid}$

。对于核函数任务，编译失败获得  $-1$ ，可执行但错误的响应获得  $+A_i^{perf}$  得  $i-5$ ；其他验证器失败获得  $-0.5$ 。正确响应获得  $-0.1$ 。一化的对数加速比，对 ArchXBench 使用中心化的断言覆盖率；当不存在分级性能信号时该项为零。遵循 GRPO (Shao 等, 2024) 使用的组相对基线，组中心化随后产生  $\bar{A}_i^{OPSD}$  包含响应  $i$  的同一提示词展开组；组内相对更优的候选获得正奖励。令  $\ell_{it}^T$  表示在  $c^*, \ell_{it}^{old}$  下标记  $i$  的冻结教师对数似然， $\ell_{it}^\theta$  表示当前学生对数似然； $sg$  表示停止梯度。三个分支向相同的截断 PPO 风格更新 (Schulman 等, 2017; Shao 等, 2024) 提供不同的有效优势：

$$= A_{i,raw}^{OPSD} - \frac{1}{|G_i|} \sum_{j \in G_i} A_{j,raw} \quad G_i$$

$$\begin{aligned} a_{it}^{B0} &= \ell_{it}^T - \ell_{it}^{old}, \\ a_{it}^{B1} &= \bar{A}_i^{OPSD}, \\ a_{it}^{B2} &= \bar{A}_i^{OPSD}((1 - \lambda) + \lambda w_{it}), \\ w_{it} &= \text{clip}\left(\exp\left(\text{sign}(\bar{A}_i^{OPSD})(\ell_{it}^T - \text{sg}[\ell_{it}^\theta])\right), 1 - \epsilon_w, 1 + \epsilon_w\right). \end{aligned} \quad (2)$$

所有三个分支共享基于 verl/HybridFlow 训练框架 (Sheng 等, 2024) 的相同实现，从而隔离学习信号。B0 探究上下文偏好能否决定更新方向。B1 用可执行结果替换该方向，并分配统一的标记奖励。B2 保持 B1 的方向不变，仅将教师差距用作正的有界标记级乘子。这种保号乘法形式适配了 RLSD (Yang 等, 2026)，后者将更新方向锚定于可验证结果，并仅使用正的教师-学生证据比来重新分配标记级奖励。因此，比较 B2 与 B1 可以隔离特权上下文奖励的增量贡献。

表 6：在匹配数据和训练预算下，从同一 SFT 检查点对 OPSD 目标进行的受控比较。B0 使用教师差距作为更新方向，B1 仅使用验证器结果，B2 用有界教师令牌信用增强 B1 的结果方向。数值越高越好。每行最佳结果加粗，次佳不同结果加下划线。N/A 表示 SFT 测量不可用。

| Benchmark   | Metric              | SFT          | B0    | B1           | B2           |
|-------------|---------------------|--------------|-------|--------------|--------------|
| VerilogEval | Spec-to-RTL avg@4   | 73.88        | 57.37 | <u>79.33</u> | <b>84.70</b> |
|             | Code-complete avg@4 | <u>81.73</u> | 56.09 | <u>79.97</u> | <b>81.90</b> |
| RTLML       | Functional avg@4    | <u>60.50</u> | 40.50 | 59.00        | <b>61.20</b> |
|             | Syntax avg@4        | <u>83.50</u> | 61.50 | 79.50        | <b>92.50</b> |
| ArchXBench  | Functional pass@1   | 22.54        | 11.27 | <u>23.94</u> | <b>36.60</b> |
|             | Syntax pass@1       | 59.15        | 29.58 | <u>60.56</u> | <b>67.61</b> |
| RealBench   | Functional pass@1   | <u>9.00</u>  | 4.67  | 6.00         | <b>26.70</b> |
|             | Syntax pass@1       | 13.67        | 10.67 | <u>14.00</u> | <b>50.00</b> |
| KernelBench | L1 correct (%)      | <u>40</u>    | 29    | <b>44</b>    | <u>40</u>    |
|             | L2 correct (%)      | 53           | 43    | <u>56</u>    | <b>60</b>    |
|             | L3 correct (%)      | 8            | 4     | <u>9</u>     | <b>14</b>    |

在表 6 报告的基准评估之前，智能体在自行分配的验证集上选择了 B2。B0 在每个有可用基线的度量上得分低于 SFT 检查点，并且在每一行上都低于两个结果导向分支，从而否定了差距导向优化。相对于 B1，B2 在 11 个报告的度量中改进了 10 个：包括两个 VerilogEval、RTLML、ArchXBench 和 RealBench 度量，以及 KernelBench L2 – L3。最大增益出现在 RealBench 语法和功能正确性上，分别提高了 36.0 和 20.7 个百分点；ArchXBench 功能 pass@1 提高了 12.66 个百分点。KernelBench L1 是唯一的回归项，从 44% 下降到 40%。这些结果证实了基于验证的 B2 选择，并表明有界教师令牌信用在 RTL 和核函数目标之间具有迁移性。

#### Research Bet: OPSD as the Sweet Spot Between SFT and RL

OPSD 在社区中引起了广泛的怀疑，因为其增益可能不稳定，但我们的团队维持了一个长期的研究赌注：通过精心设计的特权上下文，OPSD 可以占据 SFT 和 RL 之间的实用最佳点，快速修复能力差距，并为后续 RL 优化奠定更坚实的基础。这一角色很少进入前沿模型的核心训练流程。在本实验中，这一判断作为研究先验提供给智能体，智能体主导的 RSI 循环将我们的赌注转化为有效的训练方案，产生了令人满意的性能增益和实用的解决方案，我们相信这些可以为未来的模型训练提供参考。

## 4.4 RLVR：基于执行奖励的递归细化

**研究先验与决策空间。**强化学习可验证奖励阶段从离策略监督蒸馏检查点初始化策略，并通过编译和执行生成代码所获得的奖励来优化响应。人类提供的技能指定了可执行奖励族、任务属性验证器契约、不可变的原始通过/失败/未判定裁决、可用的训练基底以及计算预算。它要求判定正确性与基础设施故障处于分离状态，并保留每个阶段特定奖励标量背后的底层证据。在这些不变量内，智能体控制了表 7 中总结的决策：对于每个任务族，哪些行为构成奖励利用，硬资格门在哪里应用，可用证据是否支持二元或分级奖励，哪些故障类别可重试或不可判定，缺失监督如何进入组相对估计器，以及轨迹长度和任务混合如何预算。连续的试点和训练运行修订了奖励有效性和



优化。

表 7：暴露给智能体的强化学习可验证奖励研究技能。原始可执行裁决保持稳定，而资格、奖励适应和优化仅通过注册实验进行修订。

| Skill component                                                                                                                                                                                                                                   | Exposed primitives and invariants                                                                                                         | Agent decision space                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| 🔑 Executable verdict                                                                                                                                                                                                                              | Task-property contract, official harness profile, positive-pass evidence, and pass/fail/unjudged record                                   | Integrate profiles and decide when integrity is sufficient for online use.                                                     |
| 🛡️ Reward eligibility                                                                                                                                                                                                                             | Candidate-independent failure attribution and task-scoped exploit detectors                                                               | Identify reward exploits and determine where each hard gate applies.                                                           |
| <b>Condensed executable rule — exploit gate</b><br>if exploit detected: reward $\leftarrow$ 0<br>else: reward $\leftarrow$ task adapter(raw verdict)                                                                                              |                                                                                                                                           |                                                                                                                                |
| ⚙️ Reward adapter                                                                                                                                                                                                                                 | Versioned mapping from judged evidence to binary or graded scalar rewards                                                                 | Decide whether meaningful partial or performance ordering exists for a task family.                                            |
| 🔍 Missing supervision                                                                                                                                                                                                                             | Closed infrastructure taxonomy, bounded retry, sentinel, and masking contract                                                             | Classify failures, set stop thresholds, and prevent system faults from becoming gradients.                                     |
| <b>Condensed executable rule — unjudged contract</b><br>unjudged $\rightarrow$ sentinel outside the legitimate reward range<br>exclude from group statistics; set $A=0$ ; normalize by valid count                                                |                                                                                                                                           |                                                                                                                                |
| ⚙️ Learning and resources                                                                                                                                                                                                                         | Per-task success rates, four policy-relative strata, reward-variance diagnostics, trajectory-length telemetry, and production smoke tests | Refresh strata, schedule sampling across difficulty levels, and select the estimator, joint trajectory budget, and checkpoint. |
| <b>Condensed executable rule — resource and routing checks</b><br>budget $\leftarrow$  prompt  +  response <br>policy refresh $\Rightarrow$ re-estimate verified success $\Rightarrow$ update sampler; excessive unjudged rate $\Rightarrow$ halt |                                                                                                                                           |                                                                                                                                |

**构建策略对齐的任务课程。**智能体首先在共享可执行任务池上运行离策略监督蒸馏检查点，并将每个任务的成功率估计为获得其验证成功奖励的轨迹比例。然后，它将任务池划分为四个策略相对层级：前沿（0 – 25% 成功率）、探索（25 – 50%）、接近掌握（50 – 75%）和利用（75 – 100%）。早期训练强调接近掌握和利用任务，以维持可靠的奖励信号。随着成功率的提高，采样器转向探索和前沿任务。在强化学习可验证奖励期间，最新策略定期重复此评估；智能体刷新层级并调整采样分布。这种自动课程使训练数据与当前策略对齐，并使强化学习可验证奖励保持在现实世界可执行任务的移动能力边界附近（Zhou 等人，2025）。

4.4.1 核函数领域：任务条件奖励资格

**朴素基于执行的奖励的失败。**初始核函数试点同时接受了数值正确性与 Triton 核函数的存在性。策略找到了更廉价的奖励路径：一些响应在名义上的 Triton 包装器背后将计算委托给参考框架，而另一些则启动了一个恒等核函数，将加载的值复制到输出。两者都复现了预期输出，却没有执行所请求的计算。分级惩罚会保留对这些漏洞利用的正向引导。因此，循环将任务特定资格作为从可执行结果中分配奖励的前提条件。

**将资格范围限定在任务定义内。**运行时启动证据、恒等计算检测和框架委托检测成为硬性资格门槛。检测到的漏洞利用无论数值输出如何都获得零奖励。然而，统一的范围限定失败了。仅限 Triton 的任务要求在生成的核函数内部进行计算，而 KernelBench 允许部分算子替换，并评估正确性和加速比。因此，全局委托门槛拒绝了具有真实加速比的有效 KernelBench 13

解决方案。保留的规则将委托门槛限制在禁止委托的任务定义上。被判定且合格的核函数轨迹获得以下奖励：

$$r_i^K = \begin{cases} 1.0, & \text{correct with speedup} > 1.05\times, \\ 0.5, & \text{correct,} \\ 0.2, & \text{compiled but incorrect,} \\ 0.05, & \text{failed to compile,} \\ 0, & \text{failed an exploit-eligibility gate.} \end{cases} \quad (3)$$

中间奖励水平代表了策略进展。资格门槛防止了成功的漏洞利用在排名上超过真正的实现。

#### 4.4.2 RTL 领域：故障关闭的行为监督

**用行为有效性取代分级进展。**RTL 任务暴露了一个不同的奖励设计问题。其官方测试框架通过编译、仿真和任务特定的功能检查来定义成功，但没有提供与核函数加速比相对应的校准模拟量。仅编译无法提供朝向行为等价的部分进展证据。因此，循环为每个被判定的 RTL 轨迹保留了一个二元结果，并为缺失的判定保留了一个超出范围的值：官方测试框架明确验证行为，候选被判定且失败，基础设施重试后仍无判定。

$$r_i^{\text{RTL}} = \begin{cases} 1, \\ 0, \\ -1, \end{cases} \quad (4)$$

**诊断 RTL 执行中的错误判定。**RTL 后端的集成揭示了仿真器结果可能被误认为监督信号的两种方式。首先，自检测平台可能在未产生其显式正向判定前退出。将未出现失败消息视为成功，会奖励那些在完成检查前就终止的执行。保留的规则是故障关闭：除非事件被归类为基础设施故障，否则只有显式正向判定才获得奖励一。其次，格式错误的载荷和工具链不匹配可能在生成的设计被评估之前就导致失败。在一次 ArchXBench 集成中，测试平台在到达候选设计之前尝试打开一个缺失的输入；在 CVDP 中，仿真器版本不匹配改变了所支持的 SystemVerilog 构造。这些事件不携带关于策略质量的证据，并进入不可判定路径。附录 A.2 报告了实现此区分的测试框架、载荷和工具链控制。

#### 4.4.3 跨领域学习：掩码 GRPO 与轨迹预算

**将策略失败与基础设施失败分离。**RTL 中最清晰地暴露出的这一区别适用于两个领域。编译器崩溃、仿真超时、GPU 故障、服务请求丢失以及格式错误的载荷都可能阻止判定。将每个此类事件评分为零会系统性地惩罚受影响的任务族，并使损坏的后端与持续的模型能力差距无法区分。该循环引入了一个封闭集基础设施错误分类法：只有明确枚举的系统故障进入重试路径，而其他执行失败仍归因于生成的响应。重试后无法判定的轨迹获得哨兵值  $r_i = -1$ ，位于合法奖励区间  $[0, 1]$  之外，表示监督缺失。

**在组相对学习中掩码不可判定轨迹。**该哨兵值要求在训练器内部进行相应更改。用组均值替换不可判定的奖励会保持均值但降低估计方差，从而夸大有效兄弟样本的归一化优势。将  $-1$  保留为普通奖励会产生虚假的负梯度。该代理排除了 14

表 8：核函数与寄存器传输级任务中基于失败驱动的强化学习与可验证奖励的细化。每一行将智能体可用的观测与其诊断以及最终训练运行所保留的干预措施相关联。绝对奖励仅在相同任务的展开组内进行比较。验证器的实现细节在附录 A.2 中单独报告。

| Observed signal                                       | Diagnosis                                                                             | Retained intervention                                                                 | Role in RLVR                                           |
|-------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>Kernel-specific reward validity</b>                |                                                                                       |                                                                                       |                                                        |
| Correct output without genuine kernel computation     | Surface correctness could be achieved without performing the specified task.          | Converted launch, identity, and scoped delegation checks into hard eligibility gates. | Prevent reward exploitation.                           |
| Genuine KernelBench speedups rejected                 | A uniform delegation rule contradicted the benchmark's task definition.               | Restricted delegation gating to backends that prohibit framework computation.         | Preserve valid partial replacements.                   |
| <b>RTL-specific behavioral validity</b>               |                                                                                       |                                                                                       |                                                        |
| Testbench exited without an explicit positive verdict | Absence of a failure message was being interpreted as functional correctness.         | Required an explicit pass signal and otherwise failed closed.                         | Prevent false-positive RTL rewards.                    |
| Harness failed before evaluating the candidate        | Missing payloads or mismatched toolchains contained no evidence about policy quality. | Routed enumerated pre-candidate failures to retry and the unjudgeable sentinel.       | Prevent verifier faults from becoming RTL supervision. |
| <b>Cross-domain optimization</b>                      |                                                                                       |                                                                                       |                                                        |
| 由系统故障导致的持续零奖励：基础设施故障被学习为策略故障。增加了重试、越界哨兵值以及掩码组统计和损失。   |                                                                                       |                                                                                       | Remove spurious gradients.                             |
| 大多数真实基准验证任务在提示上限下消失                                   | 长度过滤改变了评估的寄存器传输级分布，但未使运行失效。                                                           | 在设定联合预算之前，根据实测分布提高了提示覆盖率。                                                             | 保持预期的任务分布。                                             |
| 长响应接近零奖励并触发了智能体内存不足                                   | 重复消耗了生成时间，而每个序列的长度设定了峰值内存。                                                            | 对每条轨迹联合限制提示与响应长度。                                                                     | 移除无效尾部并稳定训练。                                           |

通过组统计量和损失归一化处理不可判定样本。对于展开组  $G$ ，令  $r_i$  表示适用的核函数或 RTL 奖励， $m_i = \mathbf{1}[r_i \in [0, 1]]$ ，且  $N_G = \sum_{i \in G} m_i$ 。当  $N_G > 1$  时，带稳定器  $\epsilon_{\text{std}} > 0$  的掩码统计量和优势为

$$\begin{aligned}
 \mu_G &= \frac{1}{N_G} \sum_{i \in G} m_i r_i, \\
 \sigma_G^2 &= \frac{1}{N_G - 1} \sum_{i \in G} m_i (r_i - \mu_G)^2, \\
 A_i^{\text{RLVR}} &= m_i \frac{r_i - \mu_G}{\sigma_G + \epsilon_{\text{std}}}.
 \end{aligned} \tag{5}$$

策略损失除以有效轨迹数。 $N_G \leq 1$  的组产生零优势。由于每个展开组包含对同一提示的响应，分级核奖励和二元 RTL 奖励定义了任务内排序；其绝对值从不跨域比较。所得估计器将 GRPO 组相对更新应用于具有可执行监督的轨迹（Shao 等，2024）。

表 8 区分了领域特定奖励有效性与共享优化决策。

**诊断轨迹长度失效模式。** 后续实验表明，上下文长度同时影响任务分布和训练稳定性。在初始提示预算下，训练过滤器仅移除少量示例，但静默丢弃了 15 个 RealBench 验证划分中超过三分之二的数

因此，报告的分数量描述了最短的剩余 RTL 任务。提高提示配额恢复了缺失任务。独立增加响应上限未产生额外有用的训练信号。在数万次展开中，平均奖励从最短长度桶中的约 0.27 单调下降至中间桶的约 0.03 和接近上限的 0.02。最长尾部不包含满分响应，并重复中间推理或代码片段。

同一尾部导致了一次执行器反向内存溢出故障。微批已包含每个图形处理器一条序列，因此减小批大小无法消除由单条轨迹产生的峰值。提示与响应上限通过独立极限控制同一物理对象。智能体将其替换为全轨迹约束

$$|x_i| + |y_i| \leq B_{\text{traj}}, \quad (6)$$

其中  $x_i$  与  $y_i$  分别表示提示与响应。该分配允许长寄存器传输级提示而不悄然改变其评估分布，在有用时为短提示提供更多响应容量，并为两个领域保留每条轨迹的内存界限。它还移除了经验奖励接近零后的生成。

闭合递归强化学习验证器回路。

Together, these failure-driven revisions formed the RLVR loop.

#### Research Outlook: Internalizing Research Taste

智能体使我们认识到研究技能与研究能力属于不同层次。研究技能使假设可检验、可复现且安全，但最强的初始赌注仍来自人类；一旦有前景的方向被揭示，智能体在沿失败追踪新设计方面极为有效。下一个前沿是将研究品味本身内化到基础模型中，使智能体不仅能探索人类指定的研究计划，还能发明扩展该计划的假设。

## 5 方法论

前一节记录了智能体的完整发展轨迹，包括替代假设、失败试验以及跨数据、监督微调、离策略自蒸馏与强化学习验证器的证据驱动修订。本节将该轨迹提炼为保留配方，并呈现最终交付 iCoder 检查点的端到端训练流程。我们的最终配方运行在由寄存器传输级设计与图形处理器核任务构建的共享可执行任务池上。任务原生验证器耦合三个连续的参数更新阶段：监督微调  $\rightarrow \theta_{\text{SFT}}$  离策略自蒸馏 强化学习验证器

$$\theta_0 \xrightarrow{\text{OPSD}} \theta_{\text{OPSD}} \longrightarrow \theta_{\text{RLVR}}.$$

在监督微调期间，验证器决策定义语料库成员资格。对于每个任务  $x_i$ ，我们从 Qwen3.6-27B 基础模型抽取四个响应  $y_{i,k}^B$ 。四个样本均失败的任务进入教师展开阶段，由 DeepSeek-V4-Pro 生成四条轨迹  $y_{i,k}^T$ 。第一条通过验证的教师轨迹被完整保留，包括其推理与可执行答案。记  $v_i(y) \in \{0, 1\}$  为任务原生判定，能力差距语料库与全参数监督微调目标为

$$\begin{aligned} \mathcal{I}_{\text{SFT}} &= \left\{ i : \sum_{k=1}^4 v_i(y_{i,k}^B) = 0, \sum_{k=1}^4 v_i(y_{i,k}^T) \geq 1 \right\}, \\ k_i^* &= \min\{k : v_i(y_{i,k}^T) = 1\}, \quad \mathcal{D}_{\text{SFT}} = \{(x_i, y_{i,k_i^*}^T) : i \in \mathcal{I}_{\text{SFT}}\}, \\ \theta_{\text{SFT}} &= \arg \min_{\theta} -\mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{SFT}}} \sum_{t=1}^{|y|} \log \pi_{\theta}(y_t | x, y_{<t}), \quad \theta \text{ initialized from } \theta_0. \end{aligned} \quad (7)$$

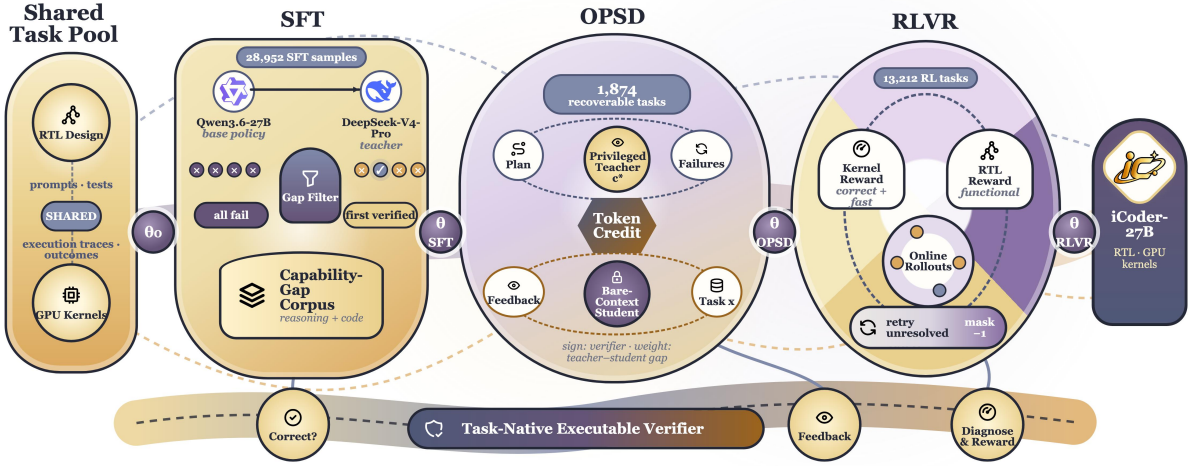


图 4：训练流程概览。共享的可执行任务池支持三个连续的参数更新阶段。监督微调（SFT）从第一条经过验证的教师轨迹构建能力差距语料库；离线特权监督蒸馏（OPSD）通过验证器签名的令牌信用将特权实验上下文迁移至裸上下文学生；强化学习可验证奖励（RLVR）将在线执行结果转化为领域感知奖励，同时屏蔽未解决的轨迹。任务原生验证器在不同阶段转换角色，从正确性检查到反馈构建，再到失败诊断与奖励分配，最终得到 iCoder-27B。

该模型相对过滤器将冷启动训练集中于基础模型 pass@4 失败且配有已验证教师轨迹的任务（Qwen Team, 2026; DeepSeek-AI, 2026）。

OPSD 在  $\theta_{\text{SFT}}$  下重新筛选，保留 1,874 个在 pass@4 时未解决、首次裸提示尝试失败、并在后续执行反馈轮次中恢复的任务。对于每个任务， $c^*$  包含经审计的仅方向计划，随后是每个失败响应及其粗略验证器反馈，终止于成功答案之前（方程 (1)）。 $\theta_{\text{SFT}}$  的冻结副本在  $(x_i, c^*)$  下对每个裸提示展开进行评分。可训练学生以  $x_i$  为条件。设  $\ell_{it}^T$ 、 $\ell_{it}^{\text{old}}$  和  $\ell_{it}^U$  分别为冻结特权教师、裸上下文展开策略和当前裸上下文学生的令牌对数似然，并设  $\tilde{A}_i^{\text{OPSD}}$  为组中心化的可执行结果优势。

$$\begin{aligned}
 w_{it} &= \text{clip}\left(\exp\left[\text{sign}(\tilde{A}_i^{\text{OPSD}})(\ell_{it}^T - \text{sg}[\ell_{it}^U])\right], 1 - \epsilon_w, 1 + \epsilon_w\right), \\
 a_{it}^{\text{B2}} &= \tilde{A}_i^{\text{OPSD}}((1 - \lambda) + \lambda w_{it}), \quad \rho_{it}(\theta) = \exp(\ell_{it}^\theta - \ell_{it}^{\text{old}}), \\
 \mathcal{L}_{\text{OPSD}}(\theta) &= -\mathbb{E}_{i,t} \left[ \min\left(\rho_{it} a_{it}^{\text{B2}}, \text{clip}(\rho_{it}, 1 - \epsilon_\pi, 1 + \epsilon_\pi) a_{it}^{\text{B2}}\right) \right].
 \end{aligned} \tag{8}$$

方程 (8) 给出了所选的 B2 更新，其中 sg 表示停止梯度。验证器优势决定更新符号，教师-学生差距定义了一个正的截断权重，用于在轨迹上重新分配令牌级信用（Schulman et al., 2017; Shao et al., 2024; Yang et al., 2026）。

RLVR 从  $\theta_{\text{OPSD}}$  继续，并将可执行证据转化为在线、任务条件化的奖励。合格的内核响应获得方程 (3) 中的分层正确性与性能奖励。RTL 响应获得方程 (4) 中的二元行为奖励。组相对优势在每个任务的展开组内计算。重试后验证仍未解决的轨迹获得哨兵值  $-1$ 。方程 (5) 中的屏蔽规则将它们从组均值、标准差、梯度和损失归一化中移除。设  $V$  表示通过屏蔽保留的已验证轨迹， $A_i^{\text{RLVR}}$  为在  $V$  上计算的组相对优势。RLVR 使用 GSPO 序列比率和



非对称截断目标 (Zheng et al., 2025) :

$$s_i(\theta) = \exp \left[ \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_{\theta}(y_{i,t} \mid x_i, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t} \mid x_i, y_{i,<t})} \right],$$

$$\mathcal{L}_{\text{GSPO}}(\theta) = -\frac{1}{|\mathcal{V}|} \sum_{i \in \mathcal{V}} \min \left( s_i(\theta) A_i^{\text{RLVR}}, \text{clip}(s_i(\theta), 1 - \epsilon_{\text{lo}}, 1 + \epsilon_{\text{hi}}) A_i^{\text{RLVR}} \right).$$
(9)

该实现将每个序列的优势值广播至其响应令牌，并采用序列均值/令牌均值聚合。熵权重为零，KL 正则化被禁用。全无效组贡献零梯度。每次展开均遵循公式 (6) 中的全轨迹预算。RLVR 产生最终的 iCoder 检查点  $\theta_{\text{RLVR}}$ 。

## 6 评估

### 6.1 实验设置

**模型与后训练数据。** 本文使用 Qwen3.6-27B 作为基础模型 (Qwen Team, 2026)。表 9 总结了每个阶段使用的训练数据。

**表 9: 按阶段划分的后训练数据。** SFT 统计完整的已验证教师轨迹，OPSD 统计可恢复的可执行任务，RLVR 统计分层验证任务。

| Stage | Training unit               | Total  | RTL            | Kernel         |
|-------|-----------------------------|--------|----------------|----------------|
| SFT   | Verified teacher trajectory | 28,952 | 18,256 (63.0%) | 10,696 (37.0%) |
| OPSD  | Recoverable executable task | 1,874  | 1,349 (72.0%)  | 525 (28.0%)    |
| RLVR  | Stratified verified task    | 13,212 | 6,058 (45.8%)  | 7,154 (54.2%)  |

**基准与度量。** 本文在 VerilogEval、RTLML、CVDP、ArchXBench 和 RealBench-Module (Liu et al., 2023; Lu et al., 2023; Pinckney et al., 2025; Purini et al., 2025; Jin et al., 2025) 上评估 RTL 生成，并在 KernelBench 和 TritonBench-G (Ouyang et al., 2025; Li et al., 2025) 上评估 GPU 核函数生成。RTL 套件涵盖从单模块补全与生成到架构级及项目集成模块任务；核函数套件涵盖单算子、算子融合、网络级工作负载及源自仓库的 Triton 核函数。本文遵循每个基准的官方任务定义与可执行测试框架，对 RTL 报告  $\text{avg}@k$  或  $\text{pass}@k$ ，对 KernelBench 报告编译/正确/快速率，对 TritonBench-G 报告正确性  $\text{pass}@1$ 。任务数量、采样预算及验证器设置见附录 A.2 (表 11)。

### 6.2 主要结果

**寄存器传输级性能比较** 表 10 显示，iCoder 在寄存器传输级测试套件中始终具有竞争力，在报告的七项度量中有四项进入前三名。它在 RTLML 功能平均@4 上达到最佳，为 68.0；在 VerilogEval 代码补全平均@4 上排名第二，为 86.0；在 CVDP 功能平均@5 上排名第二，为 44.1；在 VerilogEval 规格到 RTL 平均@4 上排名第三，为 86.3。在更复杂的基准上，尽管未领先，其 61.7 的 RealBench 语法通过@5 超过了两个 InCoder 变体、DeepSeek V4-Pro、GLM 5.2、Kimi K2.6 和 Hy3；其 49.3 的 ArchXBench 通过@1 仍高于两个 InCoder 变体、Kimi K2.6 和 Hy3。相对于 Qwen3.6-27B 基础模型，最明显的提升出现在 RTLML 上，训练后将性能从 49.6 提高到 68.0，绝对提升 18.4 个百分点。

**核函数性能比较** iCoder 同样在九项 KernelBench 度量中的七项上进入前三名，并以 20.1 的 TritonBench-G 正确性通过@1 并列最佳。

表 10：工业编码基准上的性能比较。每行最佳结果以粗体显示，次佳不同结果以下划线标出；并列结果采用相同格式。

| Benchmark   | Metric                               | iCoder<br>27B | Qwen3.6<br>27B | InCoder<br>32B | InCoder-32B<br>Thinking | DeepSeek<br>V4-Pro | GLM<br>5.2 | Kimi<br>K2.6 | GPT<br>5.5  | Claude<br>Opus-4.8 | Hy3         | Gemini<br>3.5-Flash |
|-------------|--------------------------------------|---------------|----------------|----------------|-------------------------|--------------------|------------|--------------|-------------|--------------------|-------------|---------------------|
| VerilogEval | Spec-to-RTL avg@4                    | 86.3          | 70.1           | 62.5           | 65.9                    | 69.9               | 66.0       | 72.4         | <b>90.1</b> | 82.7               | 83.8        | <u>89.1</u>         |
|             | Code-complete avg@4                  | <u>86.0</u>   | 70.8           | 58.2           | 54.2                    | 79.8               | 74.8       | 78.5         | <b>91.4</b> | 81.9               | 81.6        | <u>83.8</u>         |
| RTLML       | Functional avg@4                     | <b>68.0</b>   | 49.6           | 48.0           | 44.2                    | <u>67.5</u>        | 64.0       | 59.0         | 66.0        | 64.7               | 53.5        | 63.5                |
| CVDP        | Functional avg@5 (%)                 | <u>44.1</u>   | 33.9           | 36.9           | 30.3                    | 38.5               | 39.5       | 42.1         | 39.5        | <b>47.7</b>        | 39.7        | 29.7                |
| RealBench   | Syntax pass@5 (%)                    | 61.7          | 38.3           | 60.0           | 55.0                    | 36.7               | 43.3       | 58.3         | <u>80.0</u> | <b>83.3</b>        | 41.7        | 68.3                |
|             | Functional pass@5 (%)                | 26.7          | 16.7           | <b>46.7</b>    | <u>36.7</u>             | 16.7               | 25.0       | 25.0         | 28.3        | <u>36.7</u>        | 16.7        | 26.7                |
| ArchXBench  | Functional pass@1 (%)                | 49.3          | 35.2           | 36.6           | 29.6                    | 50.7               | 50.7       | 42.3         | <b>56.3</b> | <u>54.9</u>        | 47.9        | 50.7                |
| KernelBench | Compiled (%)                         | 95            | 87             | 88             | 85                      | 93                 | <u>96</u>  | 93           | <b>98</b>   | 95                 | 94          | 94                  |
|             | L1 Correct (%)                       | <b>61</b>     | 32             | 51             | 47                      | 32                 | 50         | 32           | 43          | <u>55</u>          | 42          | 45                  |
|             | Fast (%)                             | 25            | 12             | 18             | 18                      | 13                 | <u>26</u>  | 5            | 22          | <b>30</b>          | 21          | 23                  |
|             | Compiled (%)                         | 97            | 89             | 90             | 93                      | 91                 | 98         | 84           | <b>100</b>  | 97                 | 98          | <u>99</u>           |
|             | L2 Correct (%)                       | 74            | 28             | 65             | 63                      | 40                 | 40         | 17           | 41          | 70                 | 56          | <b>78</b>           |
|             | Fast (%)                             | <u>40</u>     | 17             | 14             | 15                      | 25                 | 30         | 7            | 24          | 37                 | 29          | <b>47</b>           |
|             | Compiled (%)                         | 90            | 86             | 60             | 60                      | 86                 | 90         | 82           | <b>100</b>  | 84                 | <u>98</u>   | <b>100</b>          |
|             | L3 Correct (%)                       | 34            | 12             | 30             | 20                      | 4                  | 30         | 18           | 38          | <u>40</u>          | 18          | <b>58</b>           |
|             | Fast (%)                             | 10            | 4              | <b>14</b>      | <u>12</u>               | 2                  | 0          | 0            | 6           | 8                  | 2           | <b>14</b>           |
|             | TritonBench-G Correctness pass@1 (%) | <b>20.1</b>   | 11.4           | 17.9           | 18.5                    | 19.0               | 19.0       | 19.0         | <u>19.5</u> | <b>20.1</b>        | <u>19.5</u> | 14.9                |

KernelBench 报告编译、正确和快速结果的通过@1。L1 和 L2 各包含 100 个任务；L3 包含 50 个任务，因此其计数转换为百分比（例如，45/17/5 变为 90/34/10）。

在 KernelBench 上，它达到最高的 L1 正确率 61%，然后在 L2 正确率和快速率上分别以 74% 和 40% 排名第二。尽管 L3 设置更难，其优势仍然可见：34% 的正确率超过了 InCoder-32B、DeepSeek V4-Pro、GLM 5.2、Kimi K2.6 和 Hy3，而 10% 的快速率超过了 GPT 5.5 和 Claude Opus-4.8。综合来看，这些结果使 iCoder 在算子级和仓库派生的核函数生成方面处于或接近领先地位，同时在全网络工作负载上保持竞争力。

6.3 强化学习可验证奖励训练动态

**从奖励到准确率。**图 5 左图和中图报告了运行过程中的验证通过率（pass@1）和每轮平均奖励。两者在整个过程中同步上升：奖励从第 20 步的 0.295 升至第 160 步的 0.446，六个比率的未加权平均值从 45.8% 升至 54.3%。提升集中在前段，大约前一百步贡献了总增益的 59%，其中 KernelBench 增益的四分之三以及 ArchXBench 的全部增益均在此阶段获得。平均奖励持续上升至第 200 步的 0.473，未出现趋于饱和的拐点。因此，执行奖励在大约一个训练轮次内转化为通过率（pass@1）。

**复杂设计上的增益。**右图显示增益在不同基准上存在差异，且在包含最复杂设计的基准上增益最大。KernelBench 要求完整的优化实现，涵盖单算子、算子融合以及整个网络架构，其通过率上升 26.0 个百分点；ArchXBench 涵盖架构级硬件设计，上升 12.7 个百分点。VerilogEval 和 RTLML 的问题较短且更简单，模型此前已分别取得 82.7% 和 64.0% 的分数，此次分别仅提升 2.9 和 4.0 个百分点。因此，强化学习可验证奖励的最大增益出现在具有更大提升空间的复杂设计上。

**跨领域兼容性。**在单一奖励接口和接近 1 的核函数与寄存器传输级比例下，所有六个后端均有所提升。核函数上的增益最大，同时每个寄存器传输级后端也均有提升，表明在共享执行奖励下具有跨领域兼容性。

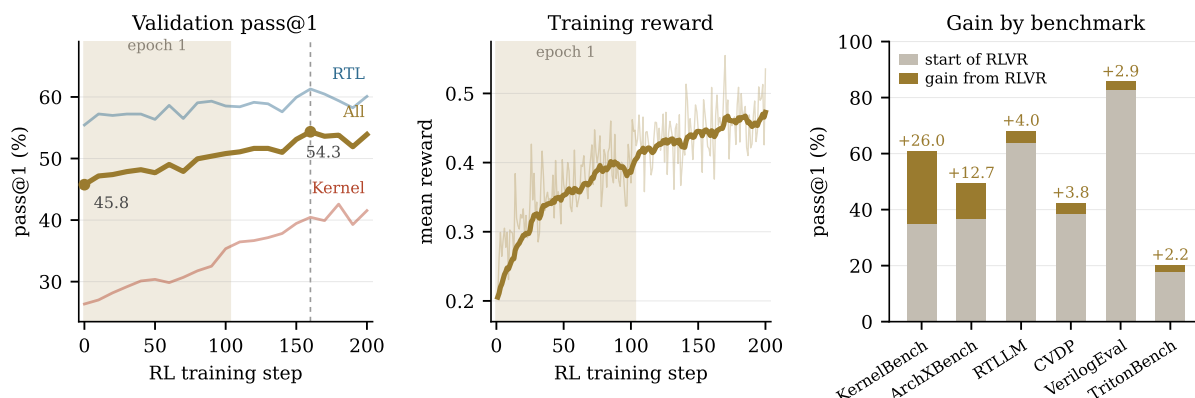


图 5：强化学习可验证奖励训练动态与结果。All 为六个验证后端的未加权平均值，Kernel 和 RTL 为各领域内的平均值；加粗奖励曲线为每步值的指数移动平均。阴影带标记第一个训练轮次，虚线标记所报告的检查点。

## 6.4 案例研究。

**强化学习验证奖励（RLVR）前后对比** 图 6 将两个验证问题与模型在 RLVR 前后的自身推理进行配对：一个来自核函数基准（KernelBench）的弗罗贝尼乌斯范数归一化，以及一个来自架构基准（ArchXBench）第二级的 32 位流水线行波进位加法器。在核函数任务上，两个版本都到达了相同的死胡同，区别在于之后的行为。RLVR 之前，模型不断重复该死胡同直到预算耗尽；RLVR 之后，模型接受该结论并绕开它，使用第二个核函数。在寄存器传输级（RTL）任务上，RLVR 之前的版本在算术的每个阶段都正确，但流水线偏移错误；而 RLVR 之后的版本陈述了对齐规则，并构建了该规则所隐含的延迟链。这些失败涉及从死胡同中恢复以及跨流水线阶段的对齐。两者都需要跨操作的协调，这与 RLVR 在复杂设计上更大的提升相一致。

### Kernel

#### Before RLVR

“……那么同一个核函数在全局同步之后执行除法？不可能。……那么同一个核函数在全局同步之后执行除法？不可能。……那么同一个核函数在全局同步之后执行除法？不可能。……”

#### After RLVR

“……我们需要等待所有块完成后再进行除法。但 Triton 核函数没有全局同步。所以我们不能在原子加法之后的同一个核函数中执行除法……我们可以启动第二个核函数。”

两个版本都到达了相同的死胡同；只有 RLVR 之后模型才离开它。重复了 971 次且未生成核函数→63% 更短，且正确。

### RTL

#### Before RLVR

“…部分和…可以被计算出来，然后只需通过流水线寄存器传递到输出…我们可以设计一个类似移位寄存器的流水线…寄存器 [7:0] sum\_low2 …寄存器 [15:0] sum\_low3 …寄存器 [23:0] sum\_low4”

#### After RLVR

“……进位输入每级延迟一个周期，因此该级的 A 和 B 切片也必须延迟相同的周期数……我们可以直接赋值 sum = {sum4\_reg, sum3\_delay1, sum2\_delay2, sum1\_delay3}”

两者都阐述了直通思想；仅在强化学习验证器之后才附加循环计数。结果滞后一个循环，50 项测试中 2 项 → 50 项中 50 项。

图 6：在两个验证问题上匹配的轨迹，采样自强化学习验证器前后的相同检查点。摘录逐字复现模型推理，寄存器传输级验证结论来自基准测试台。

**电子设计自动化作为长时程寄存器传输级优化。** 接下来测试训练后的模型能否在迭代设计循环中运行，而非仅输出单一答案。我们将 RTLScout（Arnold 等，2026）中的八个设计适配到 SimpleTES（Ye 20）的评估驱动提议-评估-优化接口。

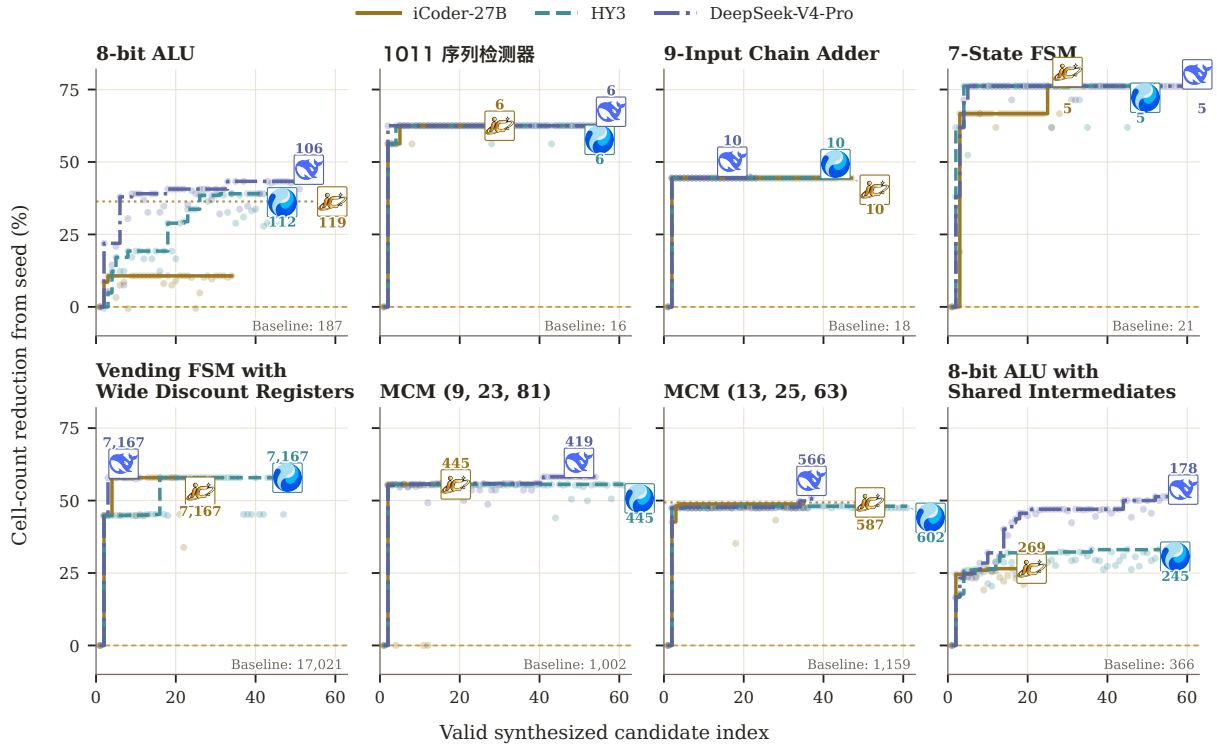
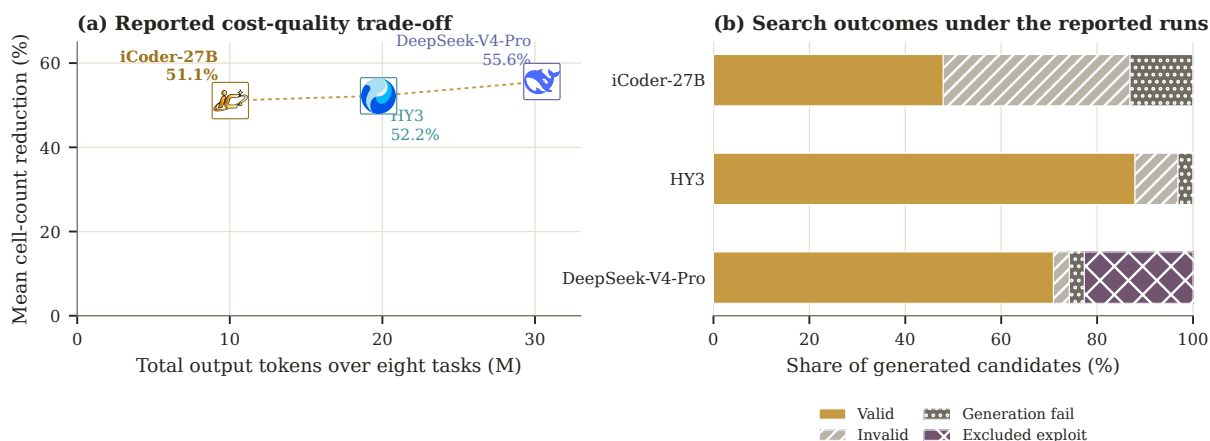


图 7：在八个改编自 RTLScout 的设计上进行评估驱动的寄存器传输级搜索。半透明点显示评估器保留的每个通过测试台验证且成功综合的候选方案；不透明阶梯线为累计最优。两者均表示为相对该任务记录种子的单元数减少量，端点图标标识模型。图标旁的粗体数字给出最终综合单元数；每个面板底部报告其基线单元数。8 位算术逻辑单元和 MCM (13, 25, 63) 上的金色虚线参考显示刷新后的 iCoder 最终结果，其候选索引不可用。其水平图标位置为视觉锚点而非候选索引。其他情况下，水平轴统计有效综合候选方案数，而非原始尝试次数或实际轮次；因此失败和被排除的候选方案不会推进该轴。

等人，2026）。该集合包含两个规格驱动的综合任务、五个寄存器传输级（RTL）重写任务和一个宽有限状态机。从每个任务的运行本地种子开始，模型在二十次迭代中每次迭代提出三个修订。每个候选方案必须通过任务的自检测平台并成功综合，然后才能按技术无关的综合后 Yosys 单元计数进行排名。单元计数越低越好；搜索得分为  $1000 / N_{\text{cell}}$ 。通过黑盒、层次保留或多模块分解逃避核算的候选方案被排除。提供的日志未记录形式等价检查，因此我们将保留的候选方案描述为经测试平台验证的，而非形式等价的。图 7 显示了每个保留的有效候选方案和累积最佳值，并针对每个任务的种子进行了归一化。其横轴统计有效的综合候选方案数量，因为记录未保留完整的尝试到轮次映射。刷新的 iCoder 导出报告了最终解决方案，但没有相应的候选索引；因此，两个改进的最终值显示为虚线金色参考，而不是附加到记录的轨迹上。有了这些更新后的结果，iCoder 在 1011 序列检测器、9 输入寄存器链加法器、7 状态有限状态机和具有宽折扣寄存器的自动售货机有限状态机上达到了共同的最佳单元计数。它在 MCM (9, 23, 81) 上与 HY3 持平，并在 MCM (13, 25, 63) 上优于 HY3。相对于 HY3，这产生了一场胜利、五场平局和两场失利。DeepSeek-V4-Pro 在其余四个任务上获得了更低的单元计数，并在其他四个任务上持平。跨任务，我们首先计算相对于每个任务种子的单元计数减少量，然后以相等权重平均这八个百分比。这避免了小单元设计主导总体结果。如图 8 所示，iCoder 的平均减少量为 51.1%，HY3 为 52.2%，



**图 8：电子设计自动化案例研究中报告的成本-质量权衡与搜索结果。** (a) 平均质量首先针对每个任务计算相对于共同记录种子的单元数减少量，然后以相等权重对八个减少量取平均。输出令牌在记录的调用中求和，作为成本代理而非货币成本。(b) 结果份额使用可用的候选日志，分母保留所有生成的候选，并将奖励利用候选作为单独的排除类别展示；刷新的最终结果元数据不改变这些计数。运行名义上每个任务使用 60 个提案；一个 DeepSeek-V4-Pro 日志包含一个额外提案。iCoder 使用 32,768 令牌的输出上限，而 HY3 和 DeepSeek-V4-Pro 使用 131,072。

DeepSeek-V4-Pro 为 55.6%，分别留下 1.1 和 4.5 个百分点的差距。它们的总输出令牌数分别为 10.03M、19.75M 和 30.45M。因此，iCoder 与 HY3 相差 1.1 个百分点，但仅使用了 51% 的输出令牌；与 DeepSeek-V4-Pro 相差 4.5 个百分点，但仅使用了 33% 的输出令牌。该比较是一项探索性案例研究，而非受控的令牌匹配基准。iCoder 使用 32,768 令牌的输出上限，而 HY3 和 DeepSeek-V4-Pro 使用 131,072；这一差异可能导致 iCoder 的不完整或无效候选比例更高。元数据还省略了生成失败的令牌使用情况，并且未保留刷新的 iCoder 最终结果、Yosys 版本或完整综合命令的完整候选历史。因此，我们将输出令牌视为报告的成本代理，在每个运行内针对记录的种子进行归一化，并且不对美元成本、墙钟时间或统计显著性作任何声明。

**在可执行反馈下的 GPU 核函数优化。** 接下来，我们在 SimpleTES 提议-评估-优化循环 (Ye 等人, 2026) 中评估三个模型用于 GPU 算子优化。归档套件包含自定义任务，带有受 RTLScout 启发的迭代搜索框架 (Arnold 等人, 2026)，以及来自归档附带的 KernelFactory 套件的样本。表 12 记录了每个算子、其计算以及归档指定的参考实现。仅当归档记录成功编译、正确性和延迟测量时，候选才进入搜索轨迹。

源记录混合了朴素相对加速比和参考相对加速比。我们重新计算每个端点为  $s_{m,t}^{\text{ref}} = \tau_t^{\text{ref}} / \tau_{m,t}$ ，其中  $\tau_t^{\text{ref}}$  是任务  $t$  的存档参考延迟。为了紧凑的  $2 \times 4$  显示，我们使用事后统计量  $g_{m,t} = (s_{m,t}^{\text{ref}} - 1) / \bar{T}_{m,t}$ ，其中  $\bar{T}_{m,t} = T_{m,t} / 10^6$  表示报告的百万输出标记数，并按  $r_t = g_{\text{iCoder},t} / \max\{g_{\text{HY3},t}, g_{\text{DeepSeek},t}\}$  对完整的三模型任务进行排序。我们保留八个最大的  $r_t$  值。GDNO 被省略，因为 HY3 没有有效端点；直方图和 Softmax 函数在完整任务中获得两个最低的  $r_t$  值。此存档标记代理仅用于选择和排序说明性面板。图 9 和图 10 显示了保留的搜索记录及其成本-性能端点。iCoder 在向量求和 ( $2.31\times$ ) 和累积求和 ( $2.96\times$ ) 上获得了最高的报告加速比。最佳存档的 iCoder 端点与最快的 SwiGLU 结果相差在 1.7% 以内，KL 散度相差 4.4%，灰度转换相差 4.8%，因果卷积 1D 相差 9.1%。在非对称矩阵乘法上差距达到 21.8%，在融合交叉熵上达到 29.4%。在此选定子集内，描述性几何-22



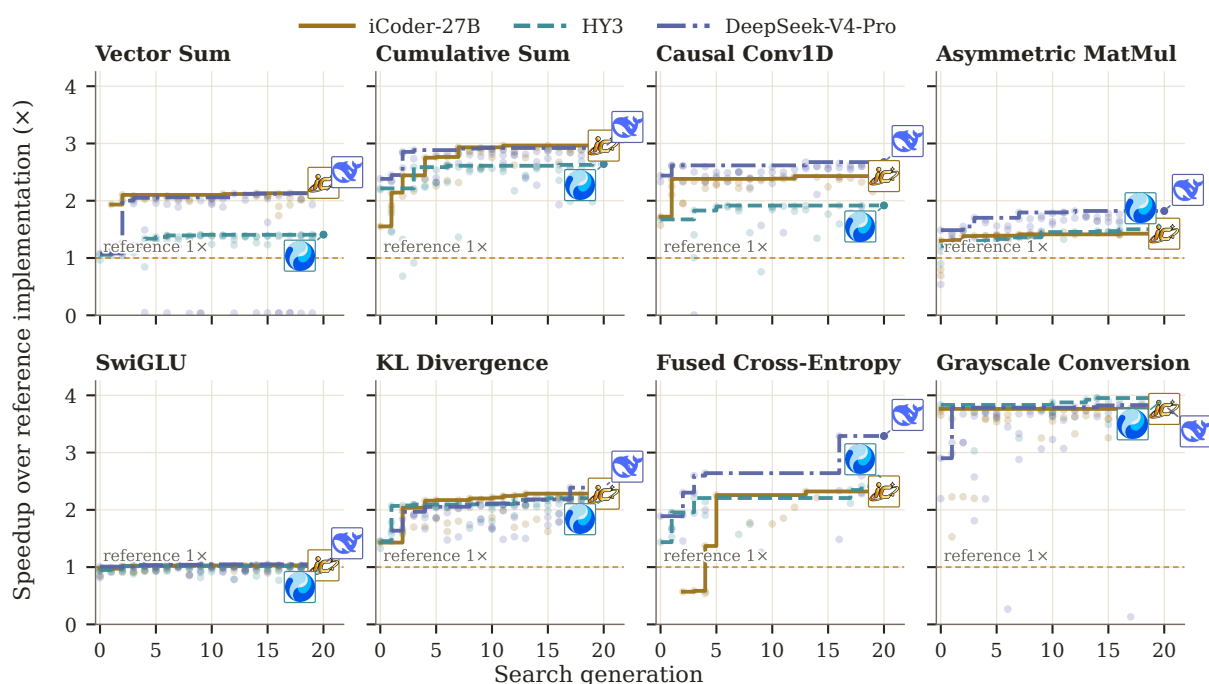


图 9：八个 GPU 算子的评估驱动搜索轨迹。半透明点是由存档标记为已编译且正确性有效的不同生成索引候选；精确重复记录已移除。不透明阶梯是累积最佳值。每个延迟都除以同一任务的存档指定参考延迟，虚线标记所得的  $1\times$  锚点。端点图标标识模型。当端点重叠时，短引线将偏移图标卡连接到其精确点。没有记录搜索生成标识符的候选不会出现在此图中。

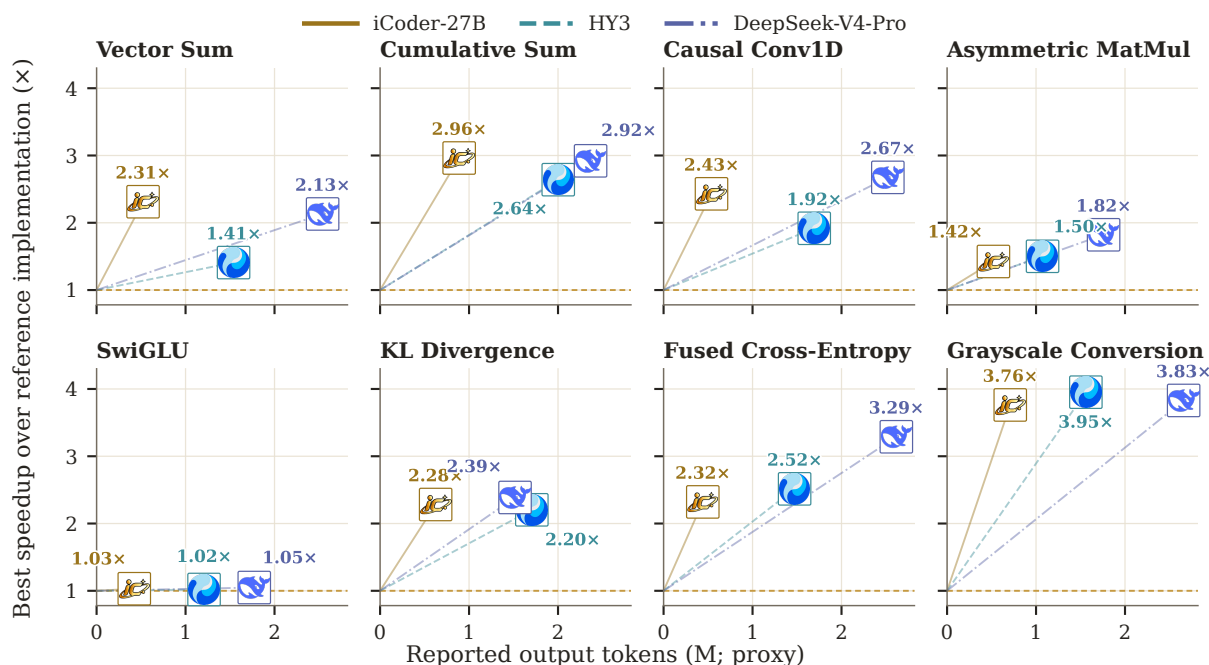


图 10：按算子报告的成本-性能权衡。每个端点将运行的总报告输出标记数与其最佳参考相对加速比配对。射线从存档指定的参考实现  $(0, 1)$  开始，因此其斜率可视化了用于案例选择的每百万报告输出标记的描述性增益。分词器、输出限制和提议计数在模型间不匹配；水平轴是搜索成本代理，而非美元成本、GPU 时间或受控计算。

平均参考相对加速比为  $2.17\times$ ,  $1.98\times$ , iCoder、HY3 和 DeepSeek-V4-Pro 分别为  $2.36\times$ 。它们记录的输出标记总数为 4.67M、12.26M 和 17.75M。根据构造,  $r_t$  在显示子集内范围从  $1.74$  到  $5.65$ ; 我们不将此范围解释为模型级效率。

这些结果并未确立预算匹配或硬件控制的效率。提案数量不同, 分词器标识、输出限制、GPU 型号和软件版本在所提供的核函数归档中均未记录。该软件包还省略了原始验证器代码、计时重复和不确定性估计。图中报告的是归档端点和令牌代理, 未涉及货币成本、GPU 时间或统计显著性的声明, 且所选子集聚合并不能表征完整的十一任务归档。

## 7 结论

递归人工智能能否真正闭合人工智能训练并改进人工智能的闭环? 我们认为这一皇冠上的明珠仍遥不可及。本文并未宣称完全自主的递归自我改进, 而是提出了一条实用路径: 人类专家与人工智能智能体共同开发一个具有前沿竞争力的工业编码模型。人类专家提供可执行的研究技能, 用于定义目标、阶段脚手架、权限边界、验证器完整性和操作流程, 同时将经验性选择留给智能体。在这一可信空间内, 智能体演化出可执行数据, 为监督微调选择能力缺口, 探索并修订 OPSD 设计, 并利用可执行反馈迭代强化学习可验证奖励的训练约束, 最终产出 27B 参数的 iCoder 模型。iCoder 在 RTLLM 上达到 68.0, 在 KernelBench L1 正确性上达到 61%, 超越了包括 DeepSeek-V4-Pro、GLM-5.2 和 Kimi-K2.6 在内的更大模型, 以及 GPT-5.5、Claude Opus 4.8 和 Gemini 3.5 Flash。它还在 TritonBench-G 上以 20.1 并列最佳结果, 并在 CVDP 上以 44.1 排名第二。更广泛地看, 这种分工为人工智能服务人工智能提供了一条可审计的路径: 人类专业知识定义安全的研究空间, 智能体积累证据、修正假设并改进模型。我们希望这有助于推动递归人工智能迈向这样的系统: 人工智能能够在训练下一代人工智能中承担日益完整的角色。

## 致谢

我们衷心感谢张嘉怡对 OPSD 先验理论发展的贡献以及对自动研究框架的指导。

## 参考文献

- Emre Can Acikgoz, Cheng Qian, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Self-improving LLM agents at test-time. *arXiv preprint arXiv:2510.07841*, 2025. doi: 10.48550/arXiv.2510.07841. <https://arxiv.org/abs/2510.07841>.
- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *International Conference on Learning Representations*, 2024. doi: 10.48550/arXiv.2306.13649.
- Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. EvoSkill: Automated skill discovery for multi-agent systems. *arXiv preprint arXiv:2603.02766*, 2026. doi: 10.48550/arXiv.2603.02766. <https://arxiv.org/abs/2603.02766>.
- Anthropic. Introducing Claude Opus 4.8. Anthropic Product Release, May 2026. <https://www.anthropic.com/news/claude-opus-4-8>.
- Felix Arnold, Ryan Amaudruz, Dimitrios Tsaras, Renzo Andri, and Lukas Cavigelli. RTLScout: Joint agentic code and synthesis optimization for efficient digital circuits. *arXiv preprint arXiv:2606.06530*, 2026. doi: 10.48550/arXiv.2606.06530. <https://arxiv.org/abs/2606.06530>.
- Henrique Assumpção, Diego Ferreira, Leandro Campos, and Fabricio Murai. Codeevolve: an open source evolutionary coding agent for algorithmic discovery and optimization. *arXiv preprint arXiv:2510.14150*, 2025.

- Joshua Brodsky, Dhruvid Kumar, Savini Kashmira, Jayanaka Danatanarayana, Jason Mars, Krisztian Flautner, and Lingjia Tang. Kernel forge: An agent harness for LLM-based generation and optimization of CUDA kernels. *arXiv preprint arXiv:2607.24762*, 2026. doi: 10.48550/arXiv.2607.24762. <https://arxiv.org/abs/2607.24762>.
- Mingguang Chen, Licheng Wang, and Bo Qu. Recursive self-improvement in AI: From bounded self-refinement to autonomous research loops. *arXiv preprint arXiv:2607.07663*, 2026. doi: 10.48550/arXiv.2607.07663. <https://arxiv.org/abs/2607.07663>.
- DeepSeek-AI. DeepSeek-V4 preview release. DeepSeek API Documentation, April 2026. <https://api-docs.deepseek.com/news/news260424/>.
- Chenhui Deng, Zhongzhi Yu, Guan-Ting Liu, Nathaniel Pinckney, Bruce Khailany, and Haoxing Ren. ACE-RTL: When agentic context evolution meets RTL-specialized LLMs. *arXiv preprint arXiv:2602.10218*, 2026. doi: 10.48550/arXiv.2602.10218. <https://arxiv.org/abs/2602.10218>.
- Tianyu Ding, Aditya Nannapaneni, Bingfan Liu, and Ling Zhang. Autonomous research agents: A survey of AI scientists and the verification gap. *arXiv preprint arXiv:2608.05179*, 2026. doi: 10.48550/arXiv.2608.05179. <https://arxiv.org/abs/2608.05179>.
- Shangheng Du, Xiangchao Yan, Jinxin Shi, Zongsheng Gao, Shiyang Feng, Zichen Liang, Boyuan Sun, Tianshuo Peng, Yifan Zhou, Xin Li, Jie Zhou, Liang He, Bo Zhang, and Lei Bai. MLEvolve: A self-evolving framework for automated machine learning algorithm discovery. *arXiv preprint arXiv:2606.06473*, 2026. doi: 10.48550/arXiv.2606.06473. <https://arxiv.org/abs/2606.06473>.
- Weimin Fu, Zeng Wang, Minghao Shao, Ramesh Karri, Muhammad Shafique, Johann Knechtel, Ozgur Sinanoglu, and Xiaolong Guo. Synthesis-in-the-loop evaluation of LLMs for RTL generation: Quality, reliability, and failure modes. *arXiv preprint arXiv:2603.11287*, 2026. doi: 10.48550/arXiv.2603.11287. <https://arxiv.org/abs/2603.11287>.
- Google DeepMind. Gemini 3.5 Flash model card. Google DeepMind Model Card, May 2026. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>.
- Jiacheng Guo, Suozhi Huang, Yunlong Gao, Zihao Li, Jason Ge, Xu Kuang, and Mengdi Wang. AQUA: Recursively self-improving quantitative trading research agents. *arXiv preprint arXiv:2608.12841*, 2026. doi: 10.48550/arXiv.2608.12841. <https://arxiv.org/abs/2608.12841>.
- Jinjun Huang, Zhongzhen Wen, Tongtong Xu, Meng Yan, Xin Xia, and Zhongxin Liu. RealisticTritonBench: A benchmark for Triton-kernel generation in real-world AI frameworks. *arXiv preprint arXiv:2608.12004*, 2026. doi: 10.48550/arXiv.2608.12004. <https://arxiv.org/abs/2608.12004>.
- Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026. doi: 10.48550/arXiv.2601.20802.
- Jaber Jaber and Osama Jaber. AutoKernel: Autonomous GPU kernel optimization via iterative agent-driven search. *arXiv preprint arXiv:2603.21331*, 2026. doi: 10.48550/arXiv.2603.21331. <https://arxiv.org/abs/2603.21331>.
- Pengwei Jin, Di Huang, Chongxiao Li, Shuyao Cheng, Yang Zhao, Xinyao Zheng, Jiaguo Zhu, Shuyi Xing, Bohan Dou, Rui Zhang, Zidong Du, Qi Guo, and Xing Hu. RealBench: Benchmarking Verilog generation models with real-world IP designs. *arXiv preprint arXiv:2507.16200*, 2025. doi: 10.48550/arXiv.2507.16200. <https://arxiv.org/abs/2507.16200>.
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement. *arXiv preprint arXiv:2607.15524*, 2026. doi: 10.48550/arXiv.2607.15524. <https://arxiv.org/abs/2607.15524>.
- Dongfang Li et al. SLAI T-Rex: Full-parameter post-training of the DeepSeek-V4 family on Ascend SuperPOD. *arXiv preprint arXiv:2607.20145*, 2026. doi: 10.48550/arXiv.2607.20145. <https://arxiv.org/abs/2607.20145>.
- Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, Haojie Wang, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. TritonBench: Benchmarking large language model capabilities for generating Triton operators. *arXiv preprint arXiv:2502.14752*, 2025. doi: 10.48550/arXiv.2502.14752. <https://arxiv.org/abs/2502.14752>.
- Hang Lin, Chongwen Liu, and Gang Yan. Large language model based agent for automated discovery in computational physics. *arXiv preprint arXiv:2606.14266*, 2026. doi: 10.48550/arXiv.2606.14266. <https://arxiv.org/abs/2606.14266>.

- Mingjie Liu, Nathaniel Pinckney, Brucec Khailany, and Haoxing Ren. VerilogEval: Evaluating large language models for Verilog code generation. *arXiv preprint arXiv:2309.07544*, 2023. doi: 10.48550/arXiv.2309.07544. <https://arxiv.org/abs/2309.07544>.
- Yao Lu, Shang Liu, Qijun Zhang, and Zhiyao Xie. RTLLM: An open-source benchmark for design RTL generation with large language model. *arXiv preprint arXiv:2308.05345*, 2023. doi: 10.48550/arXiv.2308.05345. <https://arxiv.org/abs/2308.05345>.
- Jixiang Luo. XScientist: A git-like research protocol for long-running autonomous scientific discovery. *arXiv preprint arXiv:2607.12301*, 2026. doi: 10.48550/arXiv.2607.12301. <https://arxiv.org/abs/2607.12301>.
- Alexander Novikov, Ng  n V  , Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025. doi: 10.48550/arXiv.2506.13131. <https://arxiv.org/abs/2506.13131>.
- OpenAI. Introducing GPT-5.5. OpenAI Product Release, April 2026. <https://openai.com/index/introducing-gpt-5-5/>.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher R  , and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025. doi: 10.48550/arXiv.2502.10517. <https://arxiv.org/abs/2502.10517>.
- Nathaniel Pinckney, Chenhui Deng, Chia-Tung Ho, Yun-Da Tsai, Mingjie Liu, Wenfei Zhou, Brucec Khailany, and Haoxing Ren. Comprehensive Verilog design problems: A next-generation benchmark dataset for evaluating large language models and agents on RTL design and verification. *arXiv preprint arXiv:2506.14074*, 2025. doi: 10.48550/arXiv.2506.14074. <https://arxiv.org/abs/2506.14074>.
- Suresh Purini, Siddhant Garg, Mudit Gaur, Sankalp Bhat, Sohan Mupparapu, and Arun Ravindran. Archxbench: A complex digital systems benchmark suite for llm driven rtl synthesis. In *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD (MLCAD)*, pages 1–10. IEEE, 2025.
- Qwen Team. Qwen3.6-27B. Qwen Blog, April 2026. <https://qwen.ai/blog?id=qwen3.6-27b>.
- Ben Rank, Hardik Bhatnagar, Ame  ya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. PostTrainBench: Can LLM agents automate LLM post-training? *arXiv preprint arXiv:2603.08640*, 2026. doi: 10.48550/arXiv.2603.08640. <https://arxiv.org/abs/2603.08640>.
- Aon Safdar and Mohamed Saadeldin. Long-horizon autonomous architecture research with a language-model agent: A behavioural case study. *arXiv preprint arXiv:2608.01995*, 2026. doi: 10.48550/arXiv.2608.01995. <https://arxiv.org/abs/2608.01995>.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. doi: 10.48550/arXiv.1707.06347.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. doi: 10.48550/arXiv.2402.03300.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. HybridFlow: A flexible and efficient RLHF framework. *arXiv preprint arXiv:2409.19256*, 2024. doi: 10.48550/arXiv.2409.19256.
- Rich Sutton. The bitter lesson. Incomplete Ideas, March 2019. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>. March 13, 2019.
- Jing Wang, Shang Liu, Hangan Zhou, and Zhiyao Xie. RTL-BenchMT: Dynamic maintenance of RTL generation benchmark through agent-assisted analysis and revision. *arXiv preprint arXiv:2605.15537*, 2026a. doi: 10.48550/arXiv.2605.15537. <https://arxiv.org/abs/2605.15537>.
- Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve: Recursive self-improvement of LLM agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026b. doi: 10.48550/arXiv.2607.05297. <https://arxiv.org/abs/2607.05297>.
- Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Chris Lu, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. Towards end-to-end automation of AI research. *arXiv preprint arXiv:2606.15497*, 2026. doi: 10.48550/arXiv.2606.15497. <https://arxiv.org/abs/2606.15497>.

- Chenxu Yang, Chuanyu Qin, Qingyi Si, Minghui Chen, Naibin Gu, Dingyu Yao, Zheng Lin, Weiping Wang, Jiaqi Wang, and Nan Duan. Self-distilled RLVR. *arXiv preprint arXiv:2604.03128*, 2026. doi: 10.48550/arXiv.2604.03128.
- Haotian Ye, Haowei Lin, Jingyi Tang, Yizhen Luo, Rahul Thapa, Caiyin Yang, Chang Su, Rui Yang, Ruihua Liu, Rundao Li, Zeyu Li, Pengwei Sun, Chong Gao, Dachao Ding, Guangrong He, Miaolei Zhang, Lina Sun, Wenyang Wang, Yuchen Zhong, Zhuohao Shen, Puheng Li, Pan Lu, Bianxiao Cui, Di He, Jianzhu Ma, Junfeng Li, Hexi Baoyin, Yejin Choi, Stefano Ermon, Xiaowen Chu, Tongyang Li, Yuzhi Xu, and James Zou. Structured scaling of AI discovery across diverse scientific domains. *arXiv preprint arXiv:2604.19341*, 2026. doi: 10.48550/arXiv.2604.19341. <https://arxiv.org/abs/2604.19341>.
- Peiyu Zang, Jian Tao, Jialing Zhang, Yichen Yuan, Wentao Zhang, Guang Liu, and Yonghua Lin. Kernelgenbench: A multi-source and multi-chip benchmark for llm-based kernel generation. *arXiv preprint arXiv:2607.27231*, 2026.
- Mingming Zhao, Jiqian Dong, Kangping Xu, Zadid Hasan, Chengrui Fan, Shan Jiang, Shuai Mao, Ting Lingya, Linyi Zou, Tailin Zhou, Yun Hin Chan, Wenkai Zhang, Zhanhong Zhou, Guowei Huang, Hongliang Li, Wenjing Cun, Zhitang Chen, Mingxuan Yuan, and Yanhui Geng. ScienceFlow: A long-horizon agent for ML research, scientific discovery and beyond. *arXiv preprint arXiv:2608.14354*, 2026a. doi: 10.48550/arXiv.2608.14354. <https://arxiv.org/abs/2608.14354>.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026b. doi: 10.48550/arXiv.2601.18734.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, Jingren Zhou, and Junyang Lin. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025. doi: 10.48550/arXiv.2507.18071. <https://arxiv.org/abs/2507.18071>.
- Hanzhang Zhou, Xu Zhang, Panrong Tong, Jianan Zhang, Liangyu Chen, Quyu Kong, Chenglin Cai, Chen Liu, Yue Wang, Jingren Zhou, and Steven Hoi. MAI-UI technical report: Real-world centric foundation GUI agents. *arXiv preprint arXiv:2512.22047*, 2025. doi: 10.48550/arXiv.2512.22047. <https://arxiv.org/abs/2512.22047>.
- Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses. *arXiv preprint arXiv:2608.08466*, 2026. doi: 10.48550/arXiv.2608.08466. <https://arxiv.org/abs/2608.08466>.



## A 补充材料

### A.1 任务池分布

共享可执行任务池包含 55,462 个经验证器支持的任务，并保持了监督微调语料库的领域和功能族分布。该任务池由 34,972 个寄存器传输级任务（占 63.1%）和 20,490 个图形处理器核函数任务（占 36.9%）组成。图 11 报告了相应的功能构成。

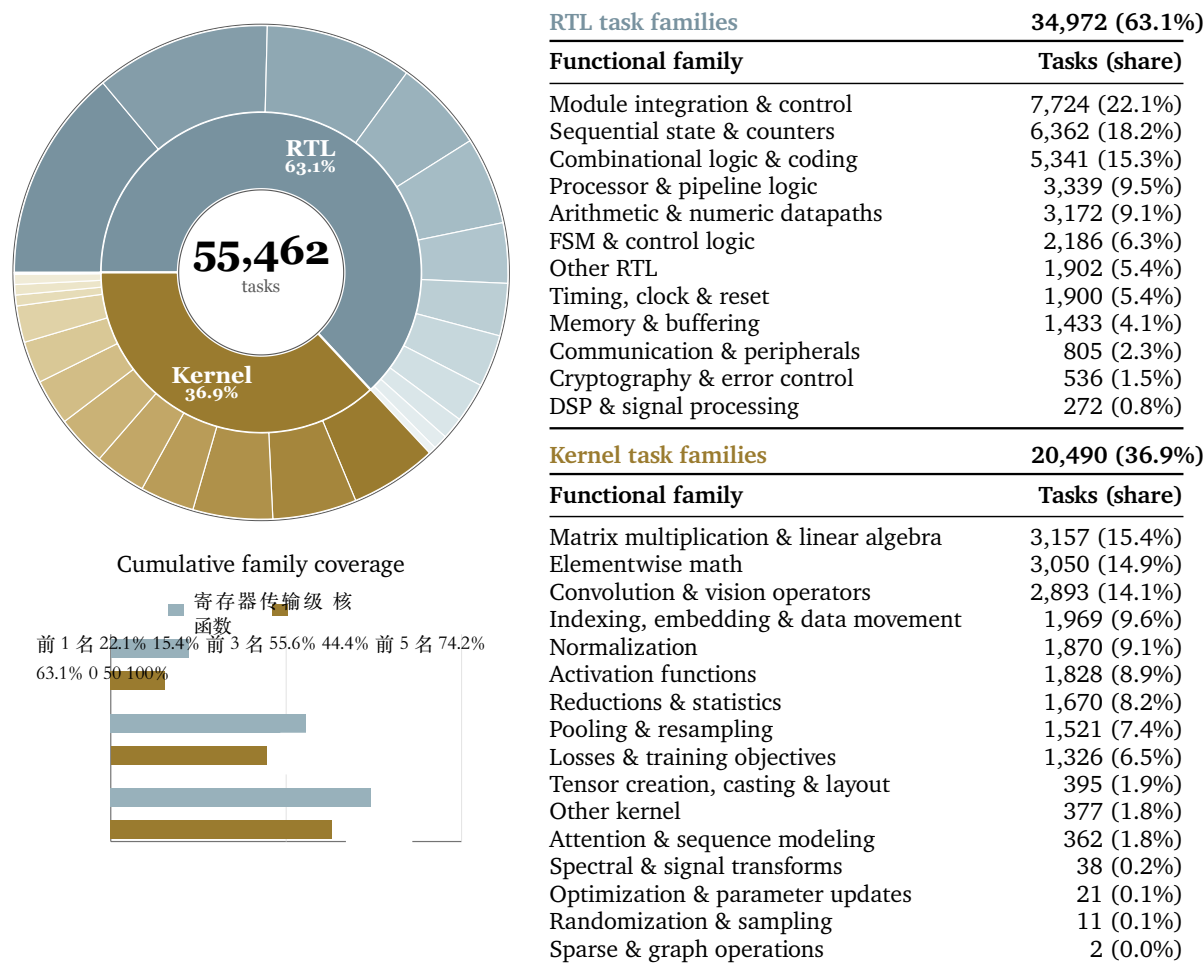


图 11：共享可执行任务池的分布。左图展示了寄存器传输级与核函数的划分以及各领域内的功能族构成；下方条形图比较了最大的一、三、五个功能族的累计覆盖率。右侧表格报告了相应的任务数量及领域内占比。

寄存器传输级覆盖率集中在模块集成与控制器化（22.1%）、时序状态与计数器（18.2%）以及组合逻辑与编码（15.3%）。核函数部分以矩阵乘法与线性代数（15.4%）、逐元素数学运算（14.9%）以及卷积与视觉算子（14.1%）为主。此外还包括损失函数与训练目标（6.5%）以及注意力机制与序列建模（1.8%），同时涵盖归一化、激活、归约、池化、索引和数据移动算子。

### A.2 评估基础设施

表 11 总结了每个基准所使用的任务清单、生成预算、采样协议、报告度量以及验证后端。对于任意采样预算  $k$ ，平均@ $k$  表示二元成功指标的平均值，通过@ $k$  表示在  $k$  个样本中至少有一个成功输出。语法度量衡量编译或展开是否成功，而功能正确性要求所有测试台 28

或数值比较均通过。

表 11：评估基础设施与基准特定设置。

| Benchmark                    | Tasks | Max tokens | Sampling ( $n$ )             | Reported metrics                | Verification                   |
|------------------------------|-------|------------|------------------------------|---------------------------------|--------------------------------|
| VerilogEval<br>Spec-to-RTL   | 156   | 49,152     | $n = 4$ , temp. 0.8          | Spec-to-RTL avg@4               | Icarus Verilog simulation      |
| VerilogEval<br>Code-complete | 156   | 49,152     | $n = 4$ , temp. 0.8          | Code-complete avg@4             | Icarus Verilog simulation      |
| RTLML                        | 50    | 49,152     | $n = 4$ , temp. 0.8          | Functional avg@4                | Icarus Verilog simulation      |
| CVDP (cid003)                | 78    | 49,152     | $n = 5$ , temp. 0.8          | Functional avg@5                | Icarus Verilog + cocotb        |
| ArchXBench v1.5              | 71    | 49,152     | $n = 1$ , temp. 0            | Functional pass@1               | Self-checking testbenches      |
| RealBench-Module             | 60    | 49,152     | $n = 5$ , temp. 0.8          | Syntax/functional pass@5        | Verilator build and simulation |
| KernelBench                  | 250   | 58,000     | $n = 1$ , temp. 0            | Compiled/correct/fast at pass@1 | Numerical comparison           |
| TritonBench-G                | 184   | 58,000     | $n = 1$ , temp. 0, zero-shot | Correctness pass@1              | Numerical comparison           |

生成过程使用单图形处理器（GPU）后端，张量并行度为一，采用半精度浮点格式（bfloat16），图形处理器内存利用率目标为 0.85。本文采用 vLLM V1 框架，搭配 VLLM\_USE\_DEEP\_GEMM=0,FlashInfer GDN 预填充后端，默认最大模型长度为 65,536 个令牌。实验在六个节点上运行，每个节点配备八块 NVIDIA H20 图形处理器。生成过程中，每个模型占用一个节点；评估过程则分片至 40 块图形处理器上执行。KernelBench 基准使用 Triton 和 CUDA 后端。其 L1、L2 和 L3 子集分别包含 100 个单算子任务、100 个算子融合任务和 50 个全网络任务。快速度量指标统计满足配置加速比阈值 ( $1.05\times$ ) 的正确核函数数量。验证过程使用各官方测试框架指定的仿真器版本：ArchXBench 使用 Icarus Verilog 11.0，VerilogEval 和 RTLML 使用 12.0，CVDP 使用 13.0，因为这些版本在 SystemVerilog 支持上存在显著差异。RealBench-Module 使用 Verilator 5.049；语法要求构建过程无任何警告输出，功能要求仿真记录无任何不匹配，因此构建时产生警告的提交在两项指标上均得零分。ArchXBench 的 0 至 3 级通过自检测测试平台进行评判，而 4 至 6 级则与基于 Makefile 的黄金模型进行对比。核函数基准在 PyTorch 2.9.1 环境下执行，搭配 Triton 3.5.1 和 CUDA 13.0。候选实现与参考实现在相同输入上运行，当所有返回张量的形状与参考一致且误差在  $10^{-2}$  容差范围内时，判定为正确。上述图形处理器和软件配置适用于七个主要基准。单独提供的迭代核函数案例研究档案未记录其图形处理器型号或软件版本，因此本文未将该配置迁移至该分析中。

### A.3 核函数案例研究算子

完整归档包含十一个算子：七个内部任务，组织在受 RTLScout 启发的迭代搜索框架（Arnold 等人，2026）之下，以及四个 KernelFactory 样本。正文可视化保留了其中八个——五个内部任务和三个 KernelFactory 样本。此来源说明描述了任务框架；所有内部算子均未作为原始 RTLScout 基准项呈现。每个任务的 reference.py 通过以下方式指定一个可信实现

基线 (baseline) 或 `ref_kernel()`. 表 12 中的延迟值是随操作员清单提供的会话测量值, 并非与硬件无关的常量。

**表 12: 完整十一任务案例研究档案的操作符定义、任务溯源及档案指定的参考实现。**正文展示根据其声明的追溯规则保留了八个任务。

| Operator             | Computation                                              | Task origin       | Reference implementation (archived latency)                       | Main display    |
|----------------------|----------------------------------------------------------|-------------------|-------------------------------------------------------------------|-----------------|
| Vector Sum           | Full reduction over all vector elements                  | In-house          | <code>torch.sum</code> (135.8 $\mu$ s)                            | Yes             |
| Cumulative Sum       | Row-wise prefix sum along dimension 1                    | In-house          | <code>torch.cumsum</code> (333.6 $\mu$ s)                         | Yes             |
| Causal Conv1D        | Left-padded convolution for sequence modeling            | In-house          | PyTorch <code>conv1d</code> with left padding (142.9 $\mu$ s)     | Yes             |
| Asymmetric MatMul    | Irregular-shape matrix multiplication                    | In-house          | PyTorch <code>matmul</code> / cuBLAS (1225.0 $\mu$ s)             | Yes             |
| Grayscale Conversion | Weighted RGB-to-luminance transform                      | In-house          | PyTorch <code>weighted tensor sum</code> (474.1 $\mu$ s)          | Yes             |
| GDNO                 | Gated Delta-Net block recurrence for linear attention    | In-house          | PyTorch <code>chunked recurrence</code> (198.4 $\mu$ s)           | No: HY3 missing |
| Histogram            | Binned counting with write contention                    | In-house          | <code>torch.histc/bincount</code> (217.5 $\mu$ s)                 | No: lower $r_t$ |
| Softmax              | Numerically stable row-wise softmax                      | KernelFactory S03 | ATen <code>F.softmax</code> , FP32 accumulation (903.8 $\mu$ s)   | No: lower $r_t$ |
| SwiGLU               | <code>SiLU(gate) <math>\odot</math> up activation</code> | KernelFactory S04 | <code>torch.compile fused expression</code> (132.0 $\mu$ s)       | Yes             |
| KL Divergence        | Temperature-scaled student-teacher distillation loss     | KernelFactory S12 | <code>torch.compile token-blocked loss</code> (997.0 $\mu$ s)     | Yes             |
| Fused Cross-Entropy  | Logits-to-loss fusion with backward-gradient readout     | KernelFactory S11 | Liger fused cross-entropy, with PyTorch fallback (1338.0 $\mu$ s) | Yes             |

## A.4 训练后可执行验证

**共同证据模型下的范围有序契约。**监督微调、离策略自蒸馏和可验证奖励强化学习通过阶段特定适配器复用任务原生执行契约。对于生成的工件及其可信任务上下文，适配器返回包含四个命名字段的证据记录：状态（通过/失败/未判定）、失败阶段、任务基准测量（如失配率、断言覆盖率或加速比）以及溯源信息（如验证器版本和载荷摘要）。可接受的预言机仍局限于工件及其可信测试框架。表 13 和表 14 按生成工件边界组织已实现的契约。寄存器传输级契约形成从模块局部完成、经独立模块、到插入受保护项目层级的模块的部分范围顺序。预言机选择与该顺序正交：在独立模块范围内，正确性可通过差分迹、显式自检判定、程序化断言或生成黄金流水线来确立。当前实现中最大的生成寄存器传输级工件是一个模块。项目集成验证在更大的依赖上下文中测试该模块。

**表 13：已实现的寄存器传输级验证契约，按生成工件边界排序。**独立模块范围内的预言机变体是由任务规范选择的替代方案；它们不代表逐步增大的设计规模。

| Verification scope        | Candidate and trusted context                                                                                                     | Native oracle                        | Required execution evidence                                                                                                                                                                             |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Module-local completion   | Candidate completion joined to a trusted module prefix and executed by a fixed harness                                            | Pinned Icarus transcript             | Exactly one target module compiles and executes; the transcript matches its trusted profile.<br>Mismatch-reporting tasks additionally require zero mismatches, a positive sample count, and no timeout. |
| Standalone module         | Complete candidate module evaluated beside trusted reference RTL                                                                  | Reference-candidate trace comparison | Both designs compile under the same generated stimulus, and their output traces agree exactly.                                                                                                          |
|                           | Complete candidate module with a trusted self-checking harness                                                                    | Explicit testbench verdict           | Simulation reaches a positive verdict and reports no failure; missing positive evidence fails closed.                                                                                                   |
|                           | Complete candidate module embedded in an allowlisted Icarus/cocotb/pytest harness                                                 | Programmatic assertions              | Every task-defined assertion completes and the test process exits successfully.                                                                                                                         |
|                           | Complete candidate module with task stimuli, reference-generation scripts, and an output comparator                               | Task-native golden pipeline          | The Makefile-driven generation, compilation, execution, and comparison pipeline completes successfully.                                                                                                 |
| Project-integrated module | Complete candidate module inserted into a protected dependency hierarchy with a trusted reference instance and stimulus generator | Differential project simulation      | The Verilator project builds and runs; its normalized transcript matches the pinned trusted profile.                                                                                                    |

**表 14：已实现的图形处理器验证契约。**两者均在任务定义的输入下将生成代码与可信参考执行进行比较，而可接受的实现边界和效率监督有所不同。

| Artifact boundary           | Candidate and trusted context                                                                               | Native oracle                                    | Accepted evidence and measurement                                                                                                                                                                     |
|-----------------------------|-------------------------------------------------------------------------------------------------------------|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Operator implementation     | Candidate device-level operator and a trusted executable reference, run with matched deterministic seeds    | Structured numerical equivalence                 | Output structures and shapes agree and tensor values satisfy the task tolerance. A completed device launch is required; prohibited framework delegation and identity stores invalidate the result.    |
| Reference-model replacement | Candidate replacement module, trusted reference model, and task-defined initialization and input generators | Repeated output equivalence under matched inputs | Correctness is checked across repeated trials. Matched timing against the reference supplies a separate speedup measurement; unchanged framework operators outside the replacement remain admissible. |

**可执行证据的阶段特定使用。**三个训练后阶段将这些契约应用于不同决策。监督微调将正确性判定作为准入门控。离策略自蒸馏保留执行阶段和任务原生测量作为反馈及失败条件经验。可验证奖励强化学习将验证结果和资格信号映射为标量奖励。

**可信测试框架与候选隔离。**验证器输入分为可信任务材料与不可信模型输出。序列化载荷具有大小限制、模式检查，并限制为相对路径。根据后端的不同，可信任务材料通过 SHA-256 允许列表、任务令牌或判定配置文件进行绑定；配置文件固定数据集修订版本、工具链版本以及预期的转录摘要。在线服务在启动时还会对其验证器模块、GPU 评估器和可信任务源树进行指纹识别。因此，缺少依赖项、配置文件不匹配或源树更改都会返回未验证状态。RTL 候选被单独提取，筛选受保护模块冲突和不安全的运行时构造，并在临时目录中在时间、内存、文件和进程限制下进行编译或执行。GPU 请求获得新的进程组和 CUDA 上下文；其结果通过一个有界的 JSON 通道传递，该通道的字段和类型在服务接受之前被规范化。

**执行证据与反利用检查。**对于 GPU 代码，运行时启动-退出钩子统计已完成的 Triton 启动次数；编译、预热和核对象索引不满足执行要求。静态检查还会标记返回未更改加载值的存储，以及在 Triton 之外保留禁止计算的框架操作。这些检查的范围由生成工件契约确定。仅算子契约禁止框架委托所请求的计算，而替换契约允许在替换计算之外使用未更改的框架算子，并根据参考执行来衡量效率。验证器将这些信号与正确性分开导出，奖励适配器在分配公式 (3) 中的核奖励之前，将相关信号作为资格门应用。

**失败归因与 RLVR 奖励构建。**只有来自可信控制平面通道的允许列表中的失败才会被标记为未验证；候选输出不能触发此状态。在有界重试之后，未解决的 RLVR 样本接收哨兵  $-1$ ，并从组统计和策略更新中移除（公式 (5)）。SFT 和 OPSD 保留其原始判定和诊断语义。RTL 奖励遵循公式 (4)。GRPO 比较对同一提示的响应，域采样混合控制每个任务族的更新频率。